Fakultät für Informatik
Wilhelm-Schickard-Institut
Arbeitsbereich
Datenbanken und Informationssysteme
Prof. Dr. Ulrich Güntzer

Eberhard Karls
Universität
Tübingen

Studienarbeit

Entwurf und Implementierung eines konfigurierbaren Wrappers für XML-basierte Informationsquellen

Ausarbeitung von
Robert Sösemann, April 2002

(c) Wilhelm-Schickard-Institut
für Informatik, 2002

Betreuer: Dipl.Inform. Andreas Ludwig

Die elektronische Version dieses Dokument liegt unter: http://www.webspace-journey.de/openproject/]

Inhaltsverzeichnis

I Zusammenfassung 5

II Einleitung / Motivation 5

1 Klassische Suchdienste 5
2 Neue Herausforderungen 6
3 Hintergrund: Projekt INFORMATION BROKER 8
4 Aufgabenstellung 9

III Grundlage: Common Query Format 11

1 Standardisiertes Termformat 12
2 Geschachtelte boolesche Ausdrücke 13
3 Weitere Features 13
4 DTD der Common Query Markup Language 14
5 Beispiele für Common Query-Anfragen 15

IV Entwurf der Konfigurationsdatei 16

1 Durchgängige Informationseinheit 19
2 Abbildung der Feldbezeichner 20
3 Problem: Widerspruch Struktur - Semantik 21
4 DTD und Beispiele einer Konfigurationsdatei 22

V XSLT oder XQuery als Anfragesprache? 25

1 Was ist XSLT ? 25
1.1 Pfadanweisungen mit XPath 28
1.2 Bedingte Ausführung und Funktionen 29
2 Was ist XQuery ? 29
3 Entscheidung für XSLT 33
4 Umsetzung von Common Queries in XSLT 34
4.1 Eventuelle Änderung der Struktur (Flattening) 35
4.2 Aufbau der XPath-Suchbedingung 36
4.3 Aufbau des XSLT-Stylesheets 38

VI Implementierung und Funktionsweise 40

1 XMLWrapper.java 41
2 XMLWrapperException.java 42
3 XMLSourceDescriptor.java und Property.java 42
4 CommonQuery.java, Term.java und Boolean.java 43
5 QueryMapper.java 46
6 XMLSourceFlattener.java 47

7 XSLTQuery.java **48**

VII Einschränkungen und Ausblick **50**

1 Nachteile von Strukturänderungen **50**
2 Einschränkung der Feldbezeichner-Domänen **50**
3 Performancegrenzen **51**
4 Ausblick **51**

VIII Anhang **52**

1 Ein- / Ausgaben und Zwischenschritte – Ein Szenario **52**
2 Literaturverzeichnis / Ressourcen **58**
3 Abbildungsverzeichnis **60**
4 Fehlermeldungen / Fehlercodes **61**
5 Dateiverzeichnis **63**

I Zusammenfassung

Das vorliegende Dokument ist die schriftliche Ausarbeitung einer Studienarbeit mit dem Thema „Entwurf und Implementierung eines konfigurierbaren Wrappers für XML-basierte Informationsquellen“.
Die Arbeit wurde im April 2002 von Robert Sösemann erstellt und von Dipl.Inform. Andreas Ludwig am Arbeitsbereich Datenbanken und Informationsysteme der Fakultät für Informatik der Universität Tübingen betreut.
Sie soll eine Motivation für die Themenstellung geben, Grundverständnis für die verwendeten Techniken vermitteln, Entwurfsentscheidungen dokumentieren und die konkrete Implementierung beschreiben.
Die XML-Wrapper-Applikation, der Java-Sourcecode, Beispielmaterial zum Testen und eine elektronische Version dieses Dokuments können unter der Webadresse http://www.webspace-journey.de/openproject heruntergeladen werden. Zur gedruckten Version dieser Ausarbeitung gehört eine CD-ROM, die ebenfalls all diese Dateien enthält. Genaue Informationen dazu finden sich im Anhang unter „Dateiverzeichnis“.

II Einleitung/Motivation

1 Klassische Suchdienste

Suchmaschinen sind nicht der Weisheit letzter Schluss. So oder ähnlich könnte ein Allgemeinplatz über die Qualität der Recherche im Internet lauten.

Bei den zwei momentan vorherrschenden Suchparadigmen – Suchmaschinen und Verzeichnisse - handelt es sich nämlich oft nur um zwei unterschiedliche Abwägungen zwischen Trefferqualität und Trefferanzahl.
Eine klassischer Suchmaschine a la Altavista ist eine Software, die von einem Ausgangspunkt (=einer Webseite) jedem Hyperlink folgt, den Text der passierten Seiten indiziert und so nach Abermillionen Schritten einen großen Teil aller Webseiten im Internet besucht und in einer Datenbasis gelisted hat. Da dieses „Crawlen“, wie das systematische Abarbeiten oft genannt wird, ganz ohne menschliches Zutun funktioniert, ist es sehr schnell und kann regelmäßig wiederholt werden.

Während also die Automatisierung einer der großen Vorteile von Suchmaschinen ist, sind die Suchtreffer oft qualitativ mangelhaft.
Das liegt daran, dass eine gelistete Webseite dann als relevant eingestuft wird, wenn die angefragten Suchbegriffe mit bestimmter Häufigkeit oder an bestimmten Stellen auf einer Seite vorkommen. Auch wenn es nahe liegt, dass in einem Text über Algorithmen oft das Wort „Algorithmen“ vorkommt, ist eine Verallgemeinerung dieser These sicherlich falsch.

Genau von dieser Verallgemeinerung gehen Suchmaschinen aber aus. Gibt der Benutzer beispielsweise als Suchbegriffe `Donald E. Knuth` und `Algorithm` ein, weil er mehr über das Algorithmenbuch dieses Autors erfahren will, werden Seiten zurückgegeben, in denen möglichst häufig und nah beieinander beide Begriffe vorkommen. Genau da liegt ein Problem. Denn dass in den erwarteten Seiten mit einer Buchkritik die beiden Worte sicher seltener vorkommen (oft nur genau einmal im Titel) als in Literaturlisten von im Web publizierten wissenschaftlichen Arbeiten zum Thema, ist fast garantiert.

Was fehlt, ist die Kenntnis der Semantik der Suchbegriffe, die ihr beim Auftreten der eben genannten Begriffe erkennen ließe, dass `Knuth` oft als Autor gesucht wird und in den Titeln seiner Bücher oft das Wort `Algorithm` vorkommt.

Diesem Mangel, also der fehlenden Kenntnis des echten Inhalts einer Webseite oder bestimmter Kombinationen von Suchbegriffen, begegnen Webverzeichnisse (z.B. Yahoo, Web.de).
Wie in einem Branchenbuch gibt es dort eine Hierarchie von Rubriken und Unter-Rubriken, in die alle gefunden Seiten thematisch eingeordnet werden.

So könnten beispielsweise Seiten, die Algorithmenbücher von Donald E. Knuth beschreiben, iin die folgende Rubrik eingeordnet werden:

`literatur:bücher:sachbücher:computer:programmierung:algorithmen`

Wissenschaftliche Publikationen zum Thema Algorithmen, die sich auf Literatur von Knuth stützen, wären dagegen eher unter dieser Rubrik zu finden:

`wissenschaft:informatik:publikationen:algorithmen`

Da Seiten nicht immer eindeutig in eine einzige Rubrik eingeordnet werden können, werden oft mehrere solcher Zuordnungen vergeben. Diese Einordnung nehmen fast ausschließlich Menschen in den Redaktionen der Webverzeichnisse oder die Personen vor, die Ihre Webseite selbst anmelden.

2 Neue Herausforderungen

Bei der Umsetzung von Websites ist eine Trendwende zu beobachten. Dynamische Websprachen wie PHP, JSP und das Datenformat XML stellen alte Suchmethoden vor neue Herausforderungen. Diese neuen Techniken schränken die jetzt schon eher mangelhaften Suchmethoden noch weiter ein.

Aus Gründen der Wiederverwendbarkeit, Aktualisierbarkeit und Personalisierbarkeit von Inhalten werden immer mehr Seiten erst zur Zeit einer Nutzeranfrage dynamisch erstellt und auf die Wünsche des Abfragers angepasst geliefert.
Ein typische Szenario ist z.B.: Ein Nutzer sucht beim Online-Buchshop Amazon ein Buch mit bestimmten Eigenschaften. Er gibt dort in HTML-Formularfeldern seine Wünsche ein.

Aus den passenden Rohdaten aus einer Datenbank und Gestaltungsschablonen (z.B. in PHP, JSP, ASP) wird von Serverkomponenten (z.B Java Servlets) die eigentliche HTML Seite „on-the-fly“ zusammengebaut und an den Browser des Anfragers geschickt.

Eine Suchmaschine kann diese Seite nicht mehr indizieren, da sie nie wirklich als Datei auf dem Server existiert. Die darin enthaltene Information ist also nicht mehr für sie zugänglich, da die Rohdaten in geschlossenen Datenbanken (z.B. DB2, Oracle) gespeichert sind und nicht statich in dem Format, das klassische Suchmaschinen verstehen – HTML.
Ein mittlerweile großer Teil des Webs, der so für Suchmaschinen unsichtbar bleibt, wird daher auch oft „Hidden Web“ genannt.

Obwohl diese Trennung von Rohdaten und ihrer konkreten Darstellung sinnvoll ist, ergibt sich daraus ein Problem. Als der Großteil der Daten im WWW noch aus statischen HTML-Seiten bestand, gab es ein einheitliches Seitenformat. Suchmaschinen konnten sich darauf einstellen, möglichst gut die Inhalte in diesem Format zu durchsuchen. Obwohl auch heute noch HTML das Format für die Verbreitung von Informationen im Web ist, sind es heute nur noch die Rohdaten, die physisch vorhanden sind und daher durchsucht werden könnten. Die zugrunde liegenden Datenbanken verwenden jedoch oft nicht standardisierte Formate, auf die ein effizienter Zugriff von außen fast unmöglich ist.
Wären diese in einem einheitlichen Standardformat, könnten sie Suchmaschinen leicht durchsuchen.

Genau in diese Richtung geht die Extensible Markup Language XML. Dokumente in diesem-Format benutzen wie HTML-Seiten eine offene, im Klartext verfasste Auszeichnungssprache, die bestimmte Textelemente mit Anfangs- und Endtags hervorhebt und somit logisch auszeichnet.
Während in HTML Teile des Textes ausgezeichnet werden, damit ein Browser weiß, wie sie visuell darzustellen sind, verwendet XML die Tags nur um die Struktur der Daten festzulegen. Eine hierarchische Struktur wird durch die Schachtelung der Tags ineinander abgebildet. Die Benennung der einzelnen Tags ist dabei – zumindest konzeptuell bedeutungslos. Sie dient lediglich der späteren Identifikation oder dem leichteren Erkennen von Semantik durch Menschen.
Um in Zukunft auch Maschinen die Bedeutung und Beziehung von Daten zu erschließen, werden semantische Erweiterungen von XML, wie das Ressource Description Framework (RDF), und Metadatenstandards (z.B. Dublin Core) entwickelt.

In der von XML verwendeten baumartigen Struktur der Daten lässt sich fast jede andere Datenmodellierung darstellen (z.B. objektorientiert, relational). Das Format und die Fülle ausgereifter Softwaretools machen eine maschinelle Be- und Verarbeitung dieser Daten sehr einfach.
Das hat in kurzer Zeit zu einer weiten Verbreitung von XML geführt. Viele Betreiber von Angeboten im Internet sind dazu übergegangen, XML als Universalformat zum Speichern und Austauschen von Daten zu verwenden, so dass bereits heute die Seiten vieler Webseiten dynamisch aus XML-Datenbeständen generiert werden. *(siehe dazu: [1], [2], [3])*

3 Hintergrund: Projekt INFORMATION BROKER

Im Rahmen eines Dissertationsprojekts am Arbeitsbereich „Datenbanken und Informationssysteme“ der Fakultät für Informatik an der Universität Tübingen wird ein sog. *Information Broker* entwickelt, der die oben beschriebenen Probleme bei der Internetrecherche umgeht, ohne dabei bereits eine konsequente und perfekte Umsetzung der neuen Technologien (z.B. rein XML- basierte Datenquellen versehen mit Metadaten) zu fordern.

Wie der Name Information Broker schon ausdrückt, fällt dem Programm die Aufgabe eines Maklers zu.
Eine Makler ist jemand, der stellvertretend für eine andere Person lästige, zeitaufwändige und schwierige Aufgaben übernimmt. Die Aufgaben bestehen meist darin, die Informationskluft zwischen zwei gegensätzlichen Interessengruppen so zu überbrücken, dass der Wunsch des Kunden zu möglichst optimalen Konditionen erfüllt wird.
Anstatt die Käufer selbst zu suchen, übernimmt z. B. für einen Wohnungsbesitzer ein Immobilienmakler diese Aufgabe. Statt sich Gedanken zu machen, wie die Aufgabe erfüllt werden soll, werden vom Kunden nur die Rahmenbedingungen (das „WAS“) festgelegt, also z.B. die gewünschte Verkaufspreishöhe und das Personenprofil der Nachfolger. Die Kentnisse der Umsetzung (also das „WIE“) liegen dabei im Verantwortungsbereich des Maklers.
Erleichtert wird die Aufgabe dem Makler durch seine professionelle Erfahrung beim Makeln eines bestimmten Gutes und den guten Kontakten zu Angebots- UND Nachfrageseite. (Idealerweise kennt der Immobilienmakler sowohl Besitzer als auch zahlreiche potentielle Kaufinteressenten.)

Analog zu diesem Beispiel funktioniert ein Information Broker. Statt Immobilien werden Informationen gemakelt.
Umso komplexer die Internetrecherche ist (z.B. Thematische Recherchen für Doktorarbeiten, Patentsuche) und umso ungeeigneter dafür die klassischen Suchhilfen erscheinen, desto öfter wird die Erfahrung menschlicher Suchprofis für Geld genutzt.
Außer ihrer überdurchschnittlich guten Kenntnisse von WWW-Suchmaschinen, beziehen sie zahlreiche andere Suchquellen (kostenpflichtige Datenbanken, Bibliotheksarchive usw.) mit in die Recherche ein. Diese umfassendere Sichtweise führt zu wesentlich besseren und aussagekräftigeren Ergebnissen.

Als schnelleres und weniger elitäres Gegenstück positioniert sich der elektronische Information Broker. Der Nutzer gibt lediglich an, was er als Ergebnis erwartet. Weder das „WIE“ noch das „WO“ interessieren ihn. All diese Fragen klärt der Broker selbst.
Um möglichst viele elektronisch verfügbare Informationsquellen mit in die Suche einzubeziehen, muss der Broker verschiedenste heterogene Datenquellen kennen. Diese werden idealerweise automatisch von den Anbietern dieser Quellen beim Information Broker registriert. Zum bisherigen Repertoire gehören statische und dynamische Websites, Bibliotheksarchive nach dem Z39.50-Standard, LDAP-Verzeichnisse und XML-Datenquellen.

Die Quellen sind insofern heterogen, dass sie die Daten auf verschiedene Arten speichern, abfragbar machen und die Ergebnisse darbieten.

So verwenden XML, Z39.50 und LDAP klare, vorhersagbare Organisationsstrukturen, während die Daten in HTML Seiten in einer für Maschinen nur schwach strukturierten Form vorliegen.
Die Struktur und die Granularität der Daten bestimmt, wie detailliert Anfragen gestellt werden können. Alle vier Gruppen verwenden unterschiedliche Protokolle (Z39.50, HTTP, FTP, Telnet, LDAP, usw.) und Querymechanismen (z.B. HTML-Formulare, LDAP Search Filters, XSLT).

Die Kenntnis über die Besonderheiten der Quelle und das Wissen wie die Nutzeranfrage auf die Quellen anzuwenden ist, haben innerhalb des Systems verschiedene sog. Wrapper-komponenten.
Diese Teilprogramme verstehen sich genau auf eine einzelne Datenquelle oder eine Gruppe von ähnlichen Quellen. Der Information Broker kennt die Details der einzelnen Quellen nicht - sie sind „gewrappt“, also verpackt und in ihren Details vor ihm verborgen. Der Broker entscheidet nur, welche Quellen für die Beantwortung der Anfrage sinnvoll sind und schickt diese in einer standardisierten Form an den jeweiligen Wrapper. Dieser muss die Anfrage dann in eine Form konvertieren, die die jeweilige Quelle versteht. Das Anfrageergebnis liefert der Wrapper ebenfalls in einer standardisierten Form an das Broker-System zurück.

Bemerkung: Jede neue Datenquelle, die anderen Regeln gehorcht als die bisherigen, benötigt also einen neuen Wrapper.

4 Aufgabenstellung

XML wird nicht nur als Universalformat zum Austausch strukturierter Daten, sondern immer öfter auch als dauerhaftes Speicherformat für große Datenmengen verwendet. Suchanfragen an datenbank-gestützte Webseiten müssen daher in Zukunft vom Information Broker nicht mehr umständlich über das Formular-Front-End mit Hilfe eines HTML-Wrappers gestellt werden, sondern können u. U. direkt an die XML-Rohdaten erfolgen.
Bei einer XML-Datenquelle handelt es sich um ein übliches XML-Dokument – meistens innerhalb einer Textdatei - in desen Struktur zahlreiche Datensätze abgespeichert sind.

Für solche XML-Datenquellen sollte im Rahmen dieser Studienarbeit ein generischer, konfigurierbarer Wrapper entworfen und implementiert werden.
Generisch deshalb, weil man im Gegensatz zu HTML-Wrappern nicht für jede konkrete XML-Quelle gesonderte Wrapper benötigt. Während man dort nämlich auf das Front-End, also die HTML-Formularfelder zugreift, um die darunter liegende Datenbank zu durchsuchen, stehen bei XML-Datenquellen direkt die Rohdaten zur Verfügung.

Zur Verdeutlichung bietet sich folgendes Beispiel an: Ein spezieller HTML-Wrapper für den Amazon Buchshop erlaubt die Abfrage der darunter liegenden Datenbank. Dies ist möglich, weil er weiß, wie z.B. das HTML-Formularfeld heißt, in das „Knuth“ eingetragen werden muss, um nach dem Autor zu suchen. Sobald sich aber bei einem Redesign die Anordnung Beschriftungen oder gar die Funktionalität des Front-Ends ändern, muss der zugehörige Wrapper ebenfalls geändert werden. Und dies, obwohl die eigentlichen Daten dieselben sind.

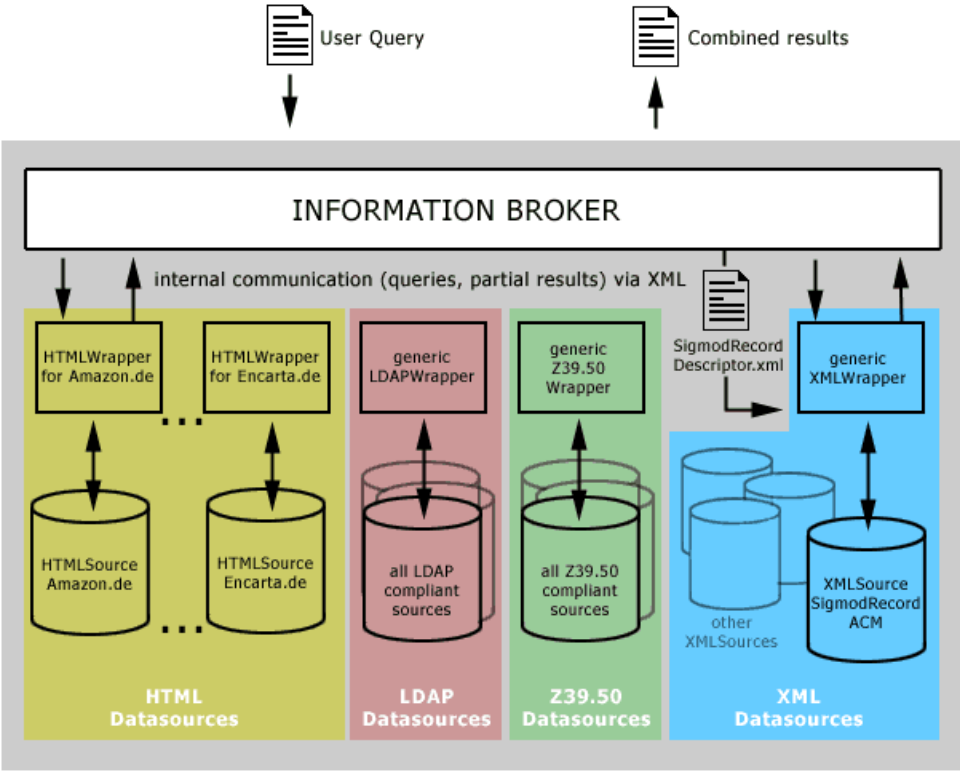


Abbildung 1: Der Information Broker und seine Komponenten

Ganz ohne konkretes Wissen über die Beschaffenheit der XML-Quelle und der durchsuchbaren Datensätze kommt der XML-Wrapper allerdings nicht aus.
Beim Anmelden einer neuen XML-Datenquelle am Information Broker muss diese durch eine Konfigurationsdatei so beschrieben werden, dass später Anfragen richtig auf diese Quelle abgebildet werden können.
Diesen Unterschied zu Wrappern für andere Datenquellen zeigt auch Abbildung 1. Während für jede beliebige LDAP- und Z39.50-Datenquelle nur ein generischer Wrapper benötigt wird, zwingen die Unterschiede in HTML-Seiten zur Entwicklung eines Wrappers für jede konkrete Anwendung. XML-Datenquellen benötigen einen speziellen Mittelweg. Der XML-Wrapper ist zwar generisch, wird allerdings für Anwendungen anhand der Konfigurationsdatei speziell auf eine Datenquelle zugeschnitten.

Der XML-Wrapper soll als eigene unabhängige Applikation implementiert werden, die bei Bedarf vom Information Broker mit folgenden Parametern aufgerufen wird:

- URL zu einer XML-basierten Konfigurationsdatei, die die gewünschte XML-Datenquelle beschreibt
- URL zu einer Nutzeranfrage in einer standardisierten Form in XML
- URL, an die die Resultate geschrieben werden sollen.

Damit der Information Broker und seine Komponenten auch über weite Entfernungen (effizient) kommunizieren können (z.B. verteiltes Rechnen über HTTP und FTP) sollen keine Dateien, sondern nur deren URLs übergeben werden.

Eine Beschreibung der Implementierung des Wrappers und seiner Architektur findet sich in Kapitel V.

Neben der Implementierungdes Wrappers in der Programmiersprache JAVA lagen die Schwerpunkte der Arbeit auf zwei Entwurfsentscheidungen:

- Beschaffenheit der Konfigurationsdateien (Welche Informationen muss sie liefern, um Nutzeranfragen abzubilden?)
- Eignung verschiedener XML-Abfragemechanismen (Welche Sprache erfüllt die Erfordernisse besser - XSLT oder XQuery?)

III Grundlage: Common Query Format

Der Information Broker verwendet eine von den Abfragemechanismen der einzelnen Datenquellen unabhängige Abfragesyntax. Alle angesprochenen Wrapper erhalten die vom Nutzer gestellte Abfrage in Form einer sog. *Common Query*. Da Queries auch gespeichert oder über das Netz als Dateien an die Einzelkomponenten verschickt werden sollen, wird extern und intern als Dateiformat eine XML-Darstellung (sie Common Query Language) benutzt. Beim Entwurf dieses Abfrageformats wurde insbesondere ein großer Mangel der vorherrschenden Suchdienste vermieden – die fehlende semantische Zuordnung der Suchbegriffe.

In unserem obigen Beispiel (Suche nach Kritiken zu Knuths Algorithmenbuch), wurde mangels semantischen Verständnisses nur nach dem Auftreten der Suchbegriffe in einem großen strukturlosen Wortcontainer gesucht – einer HTML-Seite. HTML-Seiten haben bis auf wenige logische Tags, wie z.B. <H1> oder <P>, keinerlei inhaltlich-strukturierende Syntax. Selbst wenn eine Webseite so gestaltet ist, dass menschliche Benutzer sofort die Zuordnung des Wortes „Knuth“ zur Eigenschaft „Autor“ erkennen, ist dies für Maschinen nicht möglich.

Da sie bei der Entscheidung für eine geeignete Querysprache für XML den Entscheidungsrahmen vorgibt, wird kurz der Funktionsumfang und die Syntax des benutzten Common Query-Formats umrissen.

1 Standardisiertes Termformat

Die Struktur, die HTML-Dokumente fehlt, bieten XML-Dokumente. Ähnlich zu den Tabellen und Attributen einer relationalen Datenbank kann durch beliebig schachtelbare XML-Tags und deren Attribute ein strukturiertes Datenmodell abgebildet werden.

Auszug HTML- Datenquelle :

```
<P>
 Autor: <B>Donald E. Knuth</B>
 Title: <I>The Art of Computer Programming<I><BR>
 Mit diesem Standardwerk ist bla bla
...
</P>
```

Auszug XML-Datenquelle:

```
<buchkritik>
    <autor>Donald E. Knuth</autor>
    <buch>The Art of Computer Programming</buch>
    <text>Mit diesem Standardwerk ist jeder Anwender und Wissenschaftler
    der Informationswissenschaften gut bedient. Auf 450 Seiten...</text>
</buchkritik>...
```

Während die Tags im HTML-Code lediglich die Einrückung und Darstellung im Browser beeinflussen, legen die Tags im XML-Gegenstück fest, dass es Buchkritiken gibt und diese die Attribute „autor“, „titel“ und „text“ haben.

Statt nur nach dem Auftreten einzelner Suchbegriffe fragen zu können, bietet die Common Query eine Termsyntax, also Ausdrücke im Stil der von `field relation value`.

```
autor="Donald E. Knuth"
titel="The Art of Computer Programming"
thema="Algorithmen, Programmieren"
```

Bemerkung: Syntaktisch werden Terme in der Common Query allerdings anders dargestellt.

Um die Feldbezeichner möglichst allgemeintauglich und sprach- und kontextunabhängig zu machen, werden international akzeptierte Elementnamen verwendet. Für bibliographische Daten wird das standardisierte Vokabular der Dublin Core (DC) Metadata Initiative verwendet. (s*iehe dazu [28],[29])*

Um innerhalb von XML-Dokumenten ein kontrollierbares und eindeutiges Vokabular für die Tags zu verwenden, benutzt man die Namespace-Syntax.

Dabei wird z.B. das Element `<title>` durch Voranstellen des Präfixes `dc` und einer Namespacedeklaration im Kopf des XML-Dokuments als weltweit eindeutiger Bezeichner iidentifiziert *(siehe dazu [4]).*

Die obigen Terme mit DC-Feldbezeichnern lauten:

`dc:creator=“Donald E. Knuth“`

`dc:title=“The Art of Computer Programming“`

`dc:subject=“Algorithmen, Programmieren“`

Da in den zu durchsuchenden Datenquellen die Elemente wahrscheinlich nicht `dc:creator`, sondern z.B. `autor, author, verfasser` usw. heißen, muss durch die in Kapitel IV beschriebene Konfigurationsdatei eine semantische Zuordnung vorgenommen werden. Nur so kann der Wrapper bei der Suche nach `dc:creator` z.B. das Element `<author>` durchsuchen.

2 Geschachtelte boolesche Ausdrücke

Klassische Suchmaschinen erlauben nur sehr beschränkt die Verknüpfung der Suchbegriffe durch boolesche Operatoren, wie AND, OR und NOT. Die Begriffe werden meistens standardmäßig durch AND verknüpft. Komplexere Queries durch Klammerungsolcher einzelner Ausdrücke erlaubt derzeit noch kaum eine Suchmaschine. Die Schachtelung durch Klammern ist in einer Common Query erlaubt.

Beispiel:

`(dc:creator=“Donald E. Knuth“ OR dc:creator=“C.J. Date“) AND dc:subject=“Database“`

3 Weitere Features

Um auch Zahlenwerte oder Strings zu vergleichen und zu testen, werden `<,<=,>,>=, !=` als Vergleichsoperatoren ebenfalls unterstützt. Um Groß-/Kleinschreibung, zusätzliche Leerzeichen oder andere Probleme zu umgehen, stehen praktische Funktionen wie `truncation` oder `case-sensitive` zur Verfügung. Alle erlaubten Zusatzfeatures finden sich im folgenden Abschnitt.

4 DTD der Common Query Markup Language

Die folgende Document Type Definition (DTD) beschreibt die zulässige Syntax einer Common Query.

Momentan enthält sie nur Feldbezeichner für bibliographische Daten, kann aber in Zukunft um andere Domänen erweitert werden. Im folgenden wird grundlegendes Verständnis für die einzelnen Elemente einer DTD vorausgesetzt.

```
<!-- DTD for Common Query Markup Language  Version 0.1 -->
<!-- 20.02.2002, Andreas Ludwig -->

<!ELEMENT query (term | boolean ) >
<!ELEMENT boolean ((term, term) |
                    (term, boolean) |
                    (boolean, term) |
                   (boolean, boolean)) >
<!ATTLIST boolean operator (and | or | and-not) #REQUIRED >

<!ELEMENT term EMPTY >
<!ATTLIST term value CDATA #REQUIRED
               field (dc:title | dc:creator | dc:subject |
                 dc:description |dc:publisher | dc:contributor |
                 dc:date | dc:type |dc:format | dc:identifier |
                 dc:source | dc:language |dc:relation | dc:coverage |
                 dc:rights | any) "any"
          relation (eq | lt | gt | leq | geq | neq) "eq"
          refinement CDATA #IMPLIED
          truncation (left | right | both | none) "none"
          case-sensitive (yes | no ) "no"
          encoding CDATA #IMPLIED
          modifier CDATA #IMPLIED >
```

[Abbildung 2: DTD der Common Query Markup Language]

<u>Bemerkung:</u> Die Termattribute `refinement`, `modifier` und `encoding` werden in der Implementierung des XML-Wrappers noch nicht unterstützt.

Anhand der Defaultwerte erkennt man, dass eine Query im einfachsten Fall ein einzelner Suchbegriff ist. Eine Query kann danach aus nur einem Term bestehen, bei dem nur das `value`-Attribut gesetzt ist.

So bedeutet die Query:

```
<query> <term value="Algorithms"/> </query>
```

eigentlich:

```
<query>
          <term value="Algorithms"
                field="any"
                relation="eq"
                truncation="none"
                case-sensitive="no" />
</query>
```

Mit dieser Abwärtskompatibilität ist es möglich selbst klassische Suchmaschinenanfragen fast unverändert zu benutzen.

5 Beispiele für Common Query-Anfragen

```
<query>
  <boolean op="and">
            <boolean op="or">
                <term value="Date"
                      field="dc:creator"
                      relation="eq"
                      truncation="both" />
                <term value="Donald E. Knuth"
                      field="dc:creator"
                      relation="eq" />
            </boolean>
            <term value="Database"
                  field="dc:title"
                  relation="eq"
                  truncation="both"
                  case-sentive="yes"/>
  </boolean>
</query>
```

[Abbildung 3: Common Query Beispiel 1 mit boolesch verknüpften Termen]

Es wird nach Werken gesucht, deren Autor entweder Knuth oder Date ist, und in deren Titel das Wort „Database“ vorkommt.

```
<query>
      <boolean op="and">
            <boolean op="and">
                        <term value="O'Reilly"
                              field="dc:publisher"
                              relation="eq"
                              truncation="both" />
                        <term value="XML in a nutshell"
                              field="dc:title"
                              relation="eq" />
            </boolean>
            <boolean op="and">
                        <term value="1997"
                              field="dc:date"
                              relation="geq" />
                        <term value="1999"
                              field="dc:date"
                              relation="neq" />
            </boolean>
      </boolean>
</query>
```

[Abbildung 4: Common Query Beispiel 2 mit Vergleichsrelationen]

Es wird nach Werken mit dem Titel "XML in a nutshell" vom Herausgeber O'Reilly gesucht, die seit 1997, aber nicht 1999, veröffentlicht wurden.

IV Entwurf der Konfigurationsdatei

Der XML-Wrapper kann generisch, also allgemein gültig und parametrisierbar sein, da XML standardisierte Zugriffsmechanismen hat. Was dabei konkret in der Implementierung verwendet wird, ob XSLT oder XQuery, ist in Kapitel V ausführlich dokumentiert.

Auch wenn alle XML-Datenquellen in ihrer Struktur und Syntax festen Regeln unterworfen sind, unterscheiden sie sich doch derart, dass eine Konfigurierung des XML-Wrappers unbedingt notwendig ist. Welche Metadaten minimal nötig sind, um Common Queries korrekt abzubilden, soll an folgendem Fall exemplarisch gezeigt werden. Sowohl Query als auch Datenquelle werden in den folgenden Kapiteln weiter als Standardbeispiel benutzt.

Es sollen Informationseinheiten gesucht werden, die frühestens aus dem Jahr 2000 stammen, deren Autor u.a. 'C.J. Date' ist und in deren Titel nicht das Wort 'database' vorkommt.

Als allgemein gültige Feldbezeichner eignen sich dabei:

- `dc:creator` für Autoren/Verfasser
- `dc:title` für Buch/Artikeltitel/-überschriften
- `dc:date` für Erscheinungsdaten/Jahrgänge

Die folgende Common Query setzt diese Anfrage um.

```
<query>
   <boolean op="and-not">
      <boolean op="and">
         <term value="C. J. Date"
               field="dc:creator"
               relation="eq" />
         <term value="2000"
               field="dc:date"
               relation="geq"/>
      </boolean>
      <term value="database"
            field="dc:title"
            relation="eq"
            truncation="both" />
   </boolean>
</query>
```

[Abbildung 5: Common Query Beispiel 3]

Als Datenquelle wird ein Archiv mehrerer Tausend wissenschaftlicher Artikel zum Thema „Datenbanken“ verwendet. Die amerikanische ACM stellt unter der URL http://www.acm.org/sigs/sigmod/record/xml/Record/SigmodRecord/SigmodRecord.xml diese im XML-Format zur Verfügung. Wegen der Größe der Datei (ca. 1,5 MByte und 16.000 Zeilen) ist im folgenden nur ein kleiner repräsentativer Ausschnitt (Daten sind teilweise verändert) dargestellt.

```
<SigmodRecord>
   <issue>
      <volume>1999/volume>
      <number>1</number>
      <articles>
         <article>
            <title>Remarks on Two New Theorems of Date and Fagin</title>
            <initPage>57</initPage>
            <endPage>58</endPage>
            <authors>
               <author position="00">C. J. Date</author>
```

```
            <author position="01">Ronald Fagin</author>
         </authors>
      </article>
      <article>
         <title>Architecture of Future Data Base Systems.</title>
         <initPage>30</initPage>
         <endPage>44</endPage>
         <authors>
            <author position="00">Lawrence A. Rowe</author>
         </authors>
      </article>
   ... weitere Artikel ...
   </articles>
</issue>
<issue>
   <volume>2001</volume>
   <number>3</number>
   <articles>
      <article>
         <title>A Formal Definition of the relational Model.</title>
         <initPage>18</initPage>
         <endPage>29</endPage>
         <authors>
            <author position="00">C. J. Date</author>
         </authors>
      </article>
      <article>
         <title>Database Activity at Aberdeen University.</title>
         <initPage>6</initPage>
         <endPage>6</endPage>
         <authors>
            <author position="00">Stephen Knowles</author>
         </authors>
      </article>
   </articles>
</issue>
...weitere Issues...
</SigmodRecord>
```

[Abbildung 6: Ausschnitt XML-Datenquelle SigmodRecord.xml]

Die erste Herausforderung beim Entwurf der Konfigurationsdatei ist das Fehlen einer Art SELECT-Konstrukt in den Common Queries, mit dem in SQL spezifiziert wird, welche Tupel-, bzw. Objektattribute eigentlich gesucht und ausgegeben werden sollen.

So ist mit folgendem SQL-Statement:

```
SELECT name, isbn     FROM articles
WHERE author='Date' AND date>='2000'
```

genau festgelegt, dass man am an den Werten der Attribute `name` und `isbn` aller Tupel der Relation `articles` interessiert ist, die die WHERE-Bedingung erfüllen.
Im Common Query Beispiel ist jedoch weder das FROM noch das SELECT angegeben.
Dies wäre auch nicht sinnvoll, da die Anfrage innerhalb des Information-Broker-Systems auch an andere Wrapper und damit anders aufgebaute Datenquellen geschickt wird.

Es muss also in der Konfigurationsdatei der Pfad zur eigentlichen XML-Datenquelle vermerkt werden. Um, wie gefordert, auch auf nicht-lokale Quellen zugreifen zu können, macht hier die Angabe einer weltweit eindeutigen URL Sinn.

REQUIREMENT 1 FÜR KONFIGURATIONSDATEI:
Element <source-url> spezifiziert Pfad und Zugriffsparameter der XML-Datenquelle in Form einer URL (mit protocol, port, authorization, usw.)
z.B. **<source-url>***login:pass@ftp://xmldb.com:21/Sigmod.xml***</source-url>**

1 Durchgängige Informationseinheit

Schwieriger als bei SQL gestaltet sich hingegen die Frage nach der zu betrachtenden Informationseinheit. Wo finden sich innerhalb des oft tief verschachtelten Dokumentenbaums die relevanten Datenentitäten?
Für SQL ist diese Frage einfach zu beantworten, da immer auf den Tabellen relationaler Datenbanken gearbeitet wird. Ausgegeben werden immer nur Zeilen, nämlich jene, die die Bedingungen der WHERE-Klausel erfüllen. Die in der SELECT- und WHERE-Klausel angegebenen Feldnamen sind garantiert einfache, also nicht zusammengesetzte oder verschachtelte, Attribute.

In unserem Beispiel, den ACM SigmodRecords, gibt es aber mehrere solcher – zumindest strukturellen – Entitäten. Sowohl `<issue>`, `<article>` wie auch `<authors>` enthalten weitere Elemente mit eigenen Werten. Folglich lässt sich nur durch sinnvolle Abwägung entscheiden, welches dieser Elemente und seiner Kinder eigentlich relevant ist.

Bei den SigmodRecords fällt die Wahl auf `<article>`, denn:

- `<authors>` ist als Entität nicht sinnvoll, da eine Suche nach `<author>` außer der bereits angegebenen Information, maximal alle weiteren Coautoren ausgegeben würde.
- `<issue> ist` als Entität nicht sinnvoll, da als Ergebnis zu viele nicht relevante Daten zurückgegeben würden (Missverhältnis `<issue>s` - `<article>s`). Dieses Element dient eher als Container denn als eigene Entität. Seine Eigenschaften `<volume>` und `<number>` sind aber semantisch auch Eigenschaften aller enthaltenen Artikel (vgl. später).

Man betrachtet dadurch die ACM SigmodRecords als XML-Datenquelle, die zu einer Suchanfrage passende Artikel dieses Datenbestands liefert.

REQUIREMENT 2 FÜR KONFIGURATIONSDATEI:
Element **<infounit>** spezifiziert die semantisch sinnvolle Informations-Entität, die verschiedene zu durchsuchende Eigenschaften enthält, als Elementname z.B.: **<infounit>*article*</infounit >**

Bemerkung: *Auch wenn diese Beschränkung für die meisten XML-Datenquellen ausreicht, kann nicht verallgemeinert werden, dass es nicht auch mehrere gleichwertige Informationseinheiten geben kann. Dies wird momentan aber vom implementierten XML-Wrapper nicht behandelt bzw. man muss diesen Fall mit mehreren Konfigurationsdateien für eine einzelne Datenquelle behandeln.*

2 Abbildung der Feldbezeichner

Da die in der Common Query verwendeten Feldbezeichner allgemein gültig sein sollen, stimmen diese fast nie mit den konkreten Elementnamen einer bestimmten Datenquelle überein.
So wird in der Beispielquery nach `dc:creator` gesucht, während in den SigmodRecords die Autorennamen im `<author>`-Elements stehen.
Dem Wrapper muss daher angegeben werden, welcher Common Query-Feldbezeichner auf welche Eigenschaft der Informationseinheit mappt.
Besonders behandelt werden muss der Feldbezeichner `any.` Er kann auf mehrer Elemente oder Attribute abgebildet werden und wird immer dann defaultmässig angewendet, wenn in der Query gar kein Feldbezeichner angegeben ist.

Bemerkung:
In der Konfigurationsdatei sollen nur genau die Informationen angegeben werden, die nicht automatisch durch den Wrapper oder die verwendete XML-Querysprache behandelt werden können.
Daher ist es hier nicht nötig, auf die Besonderheit des XML-Formats, Daten sowohl innerhalb von Elementen, als auch als Attributwerte darstellen zu können, einzugehen. Dies wird später gesondert behandelt.

```
REQUIREMENT 3 FÜR KONFIGURATIONSDATEI:
Element <properties> spezifiziert durch seine Kindelemente <property>
die semantisch korrekten Mappings von Query-Feldbezeichnern auf
Dokumentelemente und -attribute, z.B.:
<properties>
     <property field="dc:creator" mapto="author">
           <!-- Element kann beschreibenden Text enthalten -->
     </property>
     <property field="dc:title" mapto="title" />
     <property field="any" mapto="title, author" />
</properties>
```

Bemerkung: Bei der Verwendung mehrere Namen im `mapto`-Attribut des `any`-Feldbezeichners muss mit Kommas getrennt werden.

3 Problem: Widerspruch Struktur - Semantik

Die obige Beschreibung der Mappings weist bisher noch einen großen Mangel auf. Wie die Beispiel-Query zeigt, macht es semantisch durchaus Sinn, bei der Suche nach Artikeln auch Eigenschaften der Ausgabe, z.B. den Jahrgang, miteinzubeziehen. Das passende Element `<volume>` ist aber von der Struktur des Dokuments her kein Knoten, der von `<article>` umschlossen wird, und ist damit auch zunächst keine durchsuchbare Eigenschaft dieser Informationseinheit.

```
REQUIREMENT 4 FÜR KONFIGURATIONSDATEI:
<property> Elemente benötigen ein optionales Attribut outer, um
anzugeben, ob die Eigenschaft „ausmultipliziert“ werden muss.
<properties>
     <property field="dc:date" mapto="volume" outer="true" />
     <property field="dc:title" mapto="title" />
</properties>
```

Bemerkung: Falls auch nur eine der Eigenschaften im mehrwertigen `mapto`-Attribut von `any` eine „`outer`“-Eigenschaft ist, muss das Attribut auf `true` gesetzt werden.

Dass dies kein unüberwindbares Problem ist, sieht man leicht daran dass die Semantik der Datenquelle auch nach der folgenden Änderung der Struktur dieselbe ist.

Wie die folgende Abbildung zeigt, ist dies sowohl für Elemente wie auch Attribute nötig und möglich. Um später dieses „Ausmultiplizieren“ vor der Anwendung der Query vornehmen zu können, muss in der Konfigurationsdatei eine Änderung vorgenommen werden.
Dabei werden übergeordnete Attribute - wie `isbn` in der Abbildung - zu Attributen der Informationseinheit und übergeordnete Elemente – wie `volume` und `number` in der Abbildung – zu Kindelemente der Informationseinheit. Durch diese seperate Behandlung wird so wenig wie möglich an der ursprünglichen Modellierung geändert.

```
<issue isbn="3721234">
    <volume>1999/volume>
    <number>1</number>
    <articles>
      <article isbn="3721234">
          <volume>1999/volume>
          <number>1</number>
          <title>...</title>
          <initPage>...</initPage>
          ...
      </article>
      <article isbn="3721234">
         <volume>1999/volume>
         <number>1</number>
         <title>...</title>
         <initPage>...</initPage>
         ...
      </article>
    </articles>
</issue>
```

[Abbildung 7: Strukturänderung durch „Ausmultiplizieren“ übergeordneter Eigenschaften]

4 DTD und Beispiele einer Konfigurationsdatei

Nachfolgend finden sich die Grammatik einer Konfigurationsdatei als DTD sowie eine entsprechende Anwendung für die SigmodRecords.

```
<?xml version='1.0' encoding='UTF-8'?>
<!ELEMENT XMLSourceDescriptor (description?,
                                    source-url,
                                    infounit,
                                    resultdtd-url,
                                    properties) >

<!ELEMENT description (#PCDATA)>
```

```
<!ELEMENT source-url (#PCDATA) >
<!ELEMENT infounit (#PCDATA) >
<!ELEMENT resultdtd-url (#PCDATA) >
<!ELEMENT properties (property)+ >
<!ELEMENT property (#PCDATA)* >
<!ATTLIST property field (dc:title | dc:creator | dc:subject |
                     dc:description | dc:publisher | dc:contributor |
                     dc:date | dc:type | dc:format | dc:identifier |
                     dc:source | dc:language | dc:relation |
                     dc:coverage | dc:rights | any) #REQUIRED
                     mapto CDATA #REQUIRED
                     outer (true | false) "false" >
```

[Abbildung 8: DTD für Konfigurationsdateien]

Zusätzlich zu den schon erörterten Bestandteilen enthält die DTD einer Konfigurationsdatei noch die folgenden zusätzlichen Elemente:

- `<description>` : sinnvoll zur textuellen Kurzbeschreibung der Datenquelle für die Anzeige innerhalb der Benutzeroberfläche des Information Brokers
- `<resultdtd-url>` : falls der Information Broker die Ergebnisse des XML-Wrappers weiterverarbeiten möchte, sollte das Resultatdokument durch eine eigene DTD validiert werden können. Dies ist nötig, da das Resultat nicht mehr der Ursprungs-DTD gehorcht und sich nur mit relativ grossem Aufwand eine neue erzeugen lässt.

```
<XMLSourceDescriptor>
   <description>ACM Sigmod Articles of the year 1998-2001</description>
   <source-url>http://www.acm.org/sigmod/xml/SigmodRecord/SigmodRecord.xml
   </source-url>
   <infounit>article</infounit>
   <resultdtd-url>file:/d:/studienarbeit/dtd/SigmodRecord_result.dtd
   </resultdtd-url>
   <properties>
      <property field="dc:creator" mapto="author" />
      <property field="dc:title" mapto="title" outer="false" />
      <property field="dc:date" mapto="number" outer="true" />
      <property field="dc:identifier" mapto="volume" outer="true" />
      <property field="any" mapto="title, author" outer="true">
         If query data provides no field this field is searched by default
      </property>
   </properties>
</XMLSourceDescriptor>
```

[Abbildung 9: Beispiel-Konfigurationsdatei für SigmodRecord.xml]

V XSLT oder XQuery als Anfragesprache?

Um die allgemeinen Common Query-Anfragen an konkrete XML-Datenquellen zu stellen, benötigt der XML-Wrapper die Hilfe einer auf XML zugeschnittenen Querysprache. Diese sollte wenn möglich alle vorgestellten Funktionen direkt unterstützen.
Die Entscheidung für eine solche Sprache sollte zwischen XSLT und XQuery getroffen werden. Obwohl es diverse andere Entwicklungen und proprietäre Querysprachen für XML gibt, bestand von Beginn an der Anspruch, eine standardisierte, nicht produkt- oder firmenabhängige Variante zu verwenden.
Obwohl bisher nur XSLT den Status eines akzeptierten W3C-Standards hat, wurde mit der Entwicklung der neuen Anfragesprache XQuery die Diskussion um XML-Querysprachen derart belebt, dass XQuery ebenfalls in die Betrachtung miteinbezogen werden sollte.

Neben der kompletten und möglichst direkten Umsetzbarkeit aller Common Query Konstrukte war die leichte Verwendbarkeit innerhalb der Implementierung ein wichtiges Kriterium.

Bevor die Entscheidung erläutert wird, werden die beiden Kandidaten kurz vorgestellt.

1 Was ist XSLT ?

XSLT ist keine reine Querysprache, sondern eine Transformationssprache. XSLT ist der erste Teil der Extensible Stylesheet Language (=XSL) , die 1999 vom World Wide Web Consortium offiziell als Standard festgeschrieben wurde. Auf den anderen Teil, die XSL Formatting Objects (auch XSLFO), wird hier nicht näher eingegangen.

Das T in XSLT steht für „Transformations“, da die Hauptaufgabe einer XSLT-Anwendung die Transformation, also Umwandlung eines XML-Ausgangsdokuments in ein anderes Dokument, iist. Dabei muss das Ergebnis nicht zwangsläufig wieder XML-formatiert sein, sondern kann auch in andere SGML-konforme Markup-Formate umgewandelt werden (z.B. (X)HTML, WML). Mit Zusatzaufwand ist es sogar möglich, Dateien in fremden Formaten wie PDF, Word oder TEX zu erzeugen.

Während klassische Querysprachen wie SQL, Konstrukte zu gezielten Auswahl von klar definierten Informationseinheiten (SELECT-FROM-WHERE-Syntax) haben, verwendet XSLT ein Template-basiertes Konzept.
Dabei werden Teile eines strukturierten Dokument, so übertragen und modifiziert, dass das Resultat wieder ein strukturiertes Dokument ist. In einem XSLT-Programm (sog. Stylesheet) wird also festgelegt, was und auf welche Art in das Zieldokument übertragen werden soll.

Um nun zu definieren, dass ein bestimmter Teil eines XML-Dokument (Elemente, Attribute, usw.) übernommen werden soll, erstellt man für diesen ein Template.
Innerhalb dieser Schablone gibt man an, wie dieser Teil transformiert werden soll.

Die eigentliche Umwandlung vollzieht ein sog. XSL-Prozessor. Dies ist eine Software, die das XML-Dokument rekursiv im Top-Down-Verfahren durchläuft. Während dieses Durchlaufs wird anhand der Angaben in den Templates das Zieldokument aufgebaut.
Existiert nämlich für einen passierten Baumknoten ein Template, werden die dort vermerkten Anweisungen ausgeführt und das Resultat übernommen. Sofern innerhalb des Templates keine weiteren Templates aufgerufen werden, setzt der Prozessor seinen Durchlauf regulär fort.
Die folgenden drei XSLT-Beispiele zeigen diese Funktionsweise und wichtige XSLT-Features genauer. Alle Beispiele gehen von folgendem kurzen XML-Dokument aus:

```
<?xml version="1.0" ?>
<gruss sprache="de">
  <von versteckt="ja">Robert</von>
  <an>Welt</an>
  Hallo
</gruss>
```

[Abbildung 10: XML-Ausgangsdokument zur Erläuterung von XSLT]

Im ersten Beispiel findet sich ein Template, das auf Elemente mit dem Namen gruss passt (=matcht). Sobald im Quelldokument ein solches Element passiert wird, gibt der Prozessor den im Template eingeschlossenen Text (hier die HTML Tags) aus und fügt mit wegen der XSLT-Anweisung `<xsl:value-of>` dessen Text ein. Das Resultat dieser Transformation ist das anschliessende HTML-Dokument.

```
<xsl:stylesheet xmlns:xsl="http://www.w3c.org/1999/XSL/Transform">
     <xsl:output method="html" />
     <xsl:template match="gruss">
          <HTML>
           <HEAD><TITLE>Gruss als HTML</TITLE></HEAD>
           <BODY>
                 <xsl:value-of select="text()" />
           </BODY>
          <HTML>
     </xsl:template>
</xsl:stylesheet>
```

[Abb. 11: XSLT Beispiel 1: XML nach HTML]

```
<HTML>
           <HEAD><TITLE>Gruss als HTML</TITLE></HEAD>
           <BODY>Hallo</BODY>
</HTML>
```

XSL-Transformationen gehorchen der XML-Syntax, was ihre spätere Verwendung in Systemen, die bereits XML-fähig sind (Editoren, Parser, Validators) ,sehr einfach macht. Alle XSL-Sprachelemente werden durch die Verwendung des `xsl` Namespace-Präfixes ausgezeichnet und von verarbeitenden Programmen als solche behandelt. Alle Namespace-fremden Elemente sind für sie Teil der Ausgabe.

Selbstverständlich können ebenso XML-Dokumente einer bestimmten Struktur in XML-Dokumente anderer Struktur überführt werden. Dabei ist weder wichtig, dass diese später derselben DTD gehorchen, noch dass sämtliche Daten übertragen wurden.
Das nächste Beispiel zeigt eine solche XML-XML-Transformation. Zusätzlich enthält es ein zweites Template, das `<von>` Elemente im Ausgangsdokument in `<sender>` Elemente transformiert, nachdem es durch das Kommando `<xsl:apply-templates>` aufgerufen wurde.
Desweiteren zeigt der Zugriff auf das Attribut „sprache“ des Elements `<gruss>`, wie man Attributwerte anspricht. Im Gegensatz zu Elementen, wo die Angabe des Namens genügt, wird dort diesem zusätzlich ein „@“ vorangestellt.

```
<xsl:stylesheet xmlns:xsl="http://www.w3c.org/1999/XSL/Transform">
<xsl:output method="xml" />
     <xsl:template match="gruss">
          <greetings language="@sprache">
               <xsl:apply-templates select="von" />
               <text>
                  <xsl:value-of select="." />
               </text>
          </ greetings >
     </xsl:template>
     <xsl:template match="von">
          <sender><xsl:value-of select="text() /></sender>
     </xsl:template>
</xsl:stylesheet>
```

[Abbildung 12: XSLT Beispiel 2: XML nach XML]

Das Ergebnisdokument in XML sieht folgendermaßen aus:

```
<?xml version="1.0"?>
<greetings language="de">
<sender>Robert</sender>
<text><an>Welt</an>Hallo</text>
```

Das dritte Beispiel führt in Features von XSLT ein, die später bei der Umsetzung von Common Queries wichtig sind: XPath Anweisungen und Funktionen sowie erweiterte XSLT-Elemente.

Es handelt sich bei dem Stylesheet um einen sog. Filter, der alles bis auf bestimmte Teile ins Ergebnisdokument übernimmt. Die markierten Elemente werden nachfolgend genauer erklärt.

```
<xsl:stylesheet xmlns:xsl="http://www.w3c.org/1999/XSL/Transform">
     <xsl:output method="xml" />
     <xsl:template match="node() | @*">
          <xsl:copy>
               <xsl:apply-templates select="node() | @*" />
          </xsl:copy >
     </xsl:template>
     <xsl:template match="von">
        <xsl:if test="self::node()[normalize-space(@versteckt)='ja']">
               <sender>TOP SECRET</sender>
        </xsl:if>
     </xsl:template>
</xsl:stylesheet>
```

[Abbildung 13: XSLT Beispiel 3: Bedingte Anweisung mit xsl:if]

Das Ergebnisdokument in XML sieht folgendermaßen aus:

```
<?xml version="1.0" ?>
<gruss sprache="de">
  <sender>TOP SECRET</sender>
  <an>Welt</an>
  Hallo
</gruss>
```

1.1 Pfadanweisungen mit XPath

Um den Code besser zu verstehen, sollte man die Funktionsweise eines XSLT-Prozessors kennen.
Jede Prozessor-Implementierung hat intern einen Satz von drei oder vier sog. „Built-In-Templates“, die den rekursiven Top-Down Baumdurchlauf bewerkstelligen. Da diese Defaulttemplates auf jedes mögliche Dokument passen müssen, benötigt man eine von konkreten Dokumenten unabhängige Syntax um Baumknoten anzusprechen. Diese Funktion wird von XPath erfüllt. XPath ist erweitert die XSLT-Syntax um Pfadausdrücke, die den Betriebssystem-Dateipfaden ähnlich sind:

- / : Root bzw. Ebenentrenner (z.B.: grandfather/father/child)
- // : Shortcut über mehrerer Ebenen in der Hierarchie (z.B.: grandfather//child)
- . : der aktuelle Knoten
- .. : Zugriff auf den Vater des aktuellen Knotens (z.B. ../father)

Neben dieser abgekürzten Schreibweise kann man auch sog. Axis Specifiers verwenden. (z.B. `descendant-or-self::node()` statt //, `parent::node()` statt ..)

Im ersten Template des dritten Beispiels wird durch das Optionssymbol „|“ getrennt, sowohl auf beliebige Elemente – in XPath-Syntax „`node()`“ - und Attribute – in XPath-Syntax „`@*`“ - gematcht.

Passiert der Prozessor z.B. Elementknoten, werden diese mit `<xsl:copy>` ins Zieldokument übernommen. Dabei werden im Gegensatz zu `<xsl:copy-of select=“.“ />` nicht die nachfolgenden Knoten mitkopiert.
Damit die Anwendung des Templates rekursiv auf den nachfolgenden Knoten fortgesetzt wird, ruft man es mit `<xsl:apply-templates>` erneut auf.

1.2 Bedingte Ausführung und Funktionen

Da der Grundsatz gilt, dass konkrete Templates die allgemein gültigeren überschreiben, matcht das erste Template nicht, wenn ein `<von>`-Element passiert wird. Stattdessen wird die bedingte Ausgabe im zweiten Template ausgeführt. Mit `<xsl:if>` und seinem Attribut `test` wird überprüft, ob ein eventuell gesetztes Attribut mit dem Namen „versteckt“ den Wert „ja“ hat. Ist dies der Fall, wird statt des Namens TOP SECRET augegeben.

Bei `normalize-space()` handelt es sich um eine XSLT-Funktion, die Leerzeichen am Anfang und Ende einer Zeichenkette entfernt. XSLT bietet neben nützlichen String-Funktionen auch solche zur Behandlung von Zahlen, Knoten und Boolesche Ausdrücken.

2 Was ist XQuery ?

XSLT stammt aus einer Zeit, als XML hauptsächlich als Austauschformat zwischen verschiedenen Datenmodellen und proprietären Dateiformaten verwendet wurde. Aus klassischen Datenbanken wurden meist mittels SQL Datensätze extrahiert, in XML umgewandelt und an die Empfängerapplikation weitergegeben. Damit diese die darin enthaltenen Daten auf ihr eigenes Format abbilden konnte, wurde vorher mittels XSLT transformiert.
Mit dem Trend, XML auch als eigentliches Speicherformat für große Datenmengen zu verwenden, wurde XSLT immer mehr zweckentfremdet. Je größer aber die Datenmenge, je komplexer deren Struktur und die Abfragen wurden, um so mehr stieg auch der Bedarf an ausgereiften Queryfunktionalitäten für XML.
Mit der Zeit kam immer mehr Kritik von den Nutzern dieser Technik. Hauptangriffspunkte waren z.B., dass XSLT wegen seines deklarativen, tag-basierten Programmierparadigmas schwierig zu erlernen und umständlich in der Benutzung sei. Je komplexer die Anwendungen wurden, um so mehr wurde auch der Wunsch nach besserer Effizien laut, die es in der Welt von SQL schon lange gibt.
Beim W3C gingen zahlreiche Vorschläge (Request for Comment) und Konzepte geeigneter Querysprachen von verschiedenen Entwicklergruppen ein, und auch Firmen und Universitäten entwickelten eigene Konzepte.
Um seiner Aufgabe der Koordinierung und Förderung von internationalen Standards nachzukommen, regte das W3C eine eigene Initiative an, die die Bedürfnisse nach einer

XML- Querysprache untersuchen und umsetzen sollte*(siehe dazu [9], [24])*. Daraus entstand im Jahr 2001 der Working Draft für XQuery 1.0, einer eiegenständigen und von XSL unabhängigen Querysprache für XML.

XQuery vereint Konzepte aus unterschiedlichen Sprachen. So findet sich in XQuery für Pfadanweisungen ein leicht verändertes XPath aus XSL.
Um der Kritik an der unintuitiven Schreibweise gerecht zu werden, hat sich XQuery von der XSLT-Syntax getrennt und verwendet Sprachelemente, die stark an SQL und Konstrukte klassischer prozeduraler Programmiersprachen erinnern.

Im folgenden sind drei Beispiele für typische XQueries aufgeführt. Sie sollen nicht dem tieferen Verständnis der Sprache dienen, sondern lediglich einen Gefühl für die Unterschiede zu XSLT geben. Weiterführende Literatur zu XQuery findet sich unter [9]-[11].

```
<HTML>
<HEAD><TITLE>Gruss als HTML</TITLE></HEAD>
<BODY>
      LET $g := /gruss
      LET $v := //von
      LET $a := $g/an
      RETURN
      <B>$v sagt: $g $a</B>
</BODY>
<HTML>
```

[Abbildung 14: XQuery-Beispiel 1, XML nach HTML]

Die Ausgabe ist:

```
<HTML>
          <HEAD><TITLE>Gruss als HTML</TITLE></HEAD>
          <BODY><B>Robert sagt: Hallo Welt</B></BODY>
</HTML>
```

Auch ein XQuery Prozessor behandelt all das, was er als XQuery Code erkennt, alles andere wird in die Ausgabe übernommen (hier: die HTML Elemente).

Ausgehend von unserem XML-Ausgangsdokument aus Abbildung 10 werden oben mit drei LET-Anweisungen Variablen deklariert und mit den Werten von Elementen instanziert. Hierbei werden abgekürzte XPath-Pfadanweisungen verwendet. Anschließend werden die Werte der Variablen fett-gedruckt innerhalb des HTML-Bodys ausgegeben.
Obwohl die verwendete XPath-Syntax mit der aus XSL identisch ist, werden die Ausdrücke intern anders behandelt. Anstatt ungeordnete Knotenmengen (=nodesets) zurückzugeben, ist das Ergebnis eines XPath-Ausdrucks in XQuery ein sog. „ordered forest“ - also eine geordnete Liste von Baumknoten.

Die Nähe von XQuery zu klassischen Querysprachen wie SQL zeigt sich gut an der folgenden Query, bei der die Herausgeber von weniger als 10 oder mehr als 100 Büchern als XML ausgegeben werden.

```
<result>
          FOR $p IN distinct(document(“pub.xml“)//publisher)
          LET $b := document(“book.xml“)//book [publisher=$p]
          WHERE count($b)>100 OR count($b)<10
          RETURN <publisher>$p</publisher>
</result>
```

[Abbildung 15: XQuery-Beispiel 2: FLWR-Expression]

Nach dem typischen SELECT-FROM-WHERE-Muster werden aus der Quelle pub.xml (mit `document()` statt FROM) bestimmte Felder extrahiert (mit FOR statt SELECT) und anhand einer Bedingung in der WHERE-Klausel selektiert. Um die Ausgabe XML-wohlgeformt zu machen, ist dieser Codebereich noch innerhalb eines Elements eingeschlossen. Wegen der Anfangsbuchstaben von FOR, LET, WHERE und RETURN bezeichnet man solche Ausdrücke oft als *FLWR-Expression*.

```
FOR $s IN //schriftstueck
RETURN
          <schriftstuecke>
                $s/title,
                IF $s/@type = “zeitung“ THEN $s/verleger
                ELSE $s/autor
          </schriftstuecke>
          SORTBY (title)
```

[Abbildung 16: XQuery-Beispiel 3: Bedingte Auswertung mit IF-THEN-ELSE]

Beispiel 3 gibt eine nach Titeln sortierte Liste von Titeln von Schriftstücken und den Namen der Verantwortlichen aus. Je nach Art des Werks wird der Autor oder Herausgeber angezeigt. Für diese Unterscheidung existiert in XQuery analog zu klassischen Programmiersprachen ein IF-THEN-ELSE Konstrukt.

Verglichen mit XSLT erscheint die Syntax intuitiver und eleganter. Statt umständlicher Tags wie `<xsl:if>` können direkt die bekannten Konstrukte verwendet werden. Anstelle der gewöhnungsbedürftigen Templates können mit LET und FOR Knoten direkt ausgewählt und ausgegeben werden. Außerdem bietet XQuery viele bekannte und praktische Features, die in XSLT nur mit großem Aufwand oder gar nicht umgesetzt werden können. Dazu gehören z.B. selbstdefinierbare Funktionen, Quantifier wie SOME, ANY und ausgereifte Sortier- und Gruppierfunktionen. Zusätzlich bringt XQuery einen wesentlich umfangreicheren Satz an vordefinierten Hilfsfunktionen mit.

Trotz all dieser Vorteile hat die Ankündigung eines ersten Release-Candidates durch das W3C in der Entwicklergemeinde eine heftige Diskussion über die Qualität und den Nutzen von XQuery hervorgerufen.
Der größte Vorwurf dabei ist, mit XQuery das Rad - ohne echte Verbesserungen – neu erfunden zu haben. Vor allem namhafte Entwickler und Kenner der Szene wie Michael Kay (Entwickler des SAXON-Prozessors) und Evan Lenz kritisieren die starke Überdeckung, die eine Koexistenz beider Sprachen unsinnig erscheinen lässt.

Wie stark die Ähnlichkeiten sind, beweist Evan Lenz in seinem Artikel „XQuery: Reinventing the wheel“ [14]. Anhand einer Gegenüberstellung von XQueries und ihren XSLT-Gegenstücken führt Lenz vor, dass alle Möglichkeiten von XQuery in XSLT ihre Entsprechung finden. Obwohl einige der dort verwendeten Queries (Test-UseCase vom W3C [27]) in XQuery wesentlich eleganter und kürzer gelöst wurden, rechtfertigt dies Lenz zufolge keinen eigenen Standard.

Außer kosmetischen Änderungen an der Syntax habe auch XQuery noch die selben Mängel in Effizienz und Alltagstauglichkeit wie XSLT. Diese liegen laut Michael Kay allerdings gar nicht an schlechten Prozessorimplementierungen oder am Konzept von XSLT, sondern in der noch fehlenden mathematischen Theorie über das von XML verwendete Datenmodell.
In den Artikeln [12]-[14] wird gefordert, XSLT konsequent und evolutionär weiterzuentwickeln, statt einen zweiten minimal verbesserten Standard zu verabschieden. So erscheint vielen die Trennung von der XML-Syntax als Rückschritt in Zeiten inkompatibler Datenformate, auch wenn diese begrüßenswerte Verbesserungen mit sich bringt.

3 Entscheidung für XSLT

Bei der Wahl einer geeigneten Querysprache für den XML-Wrapper waren zwei Dinge ausschlaggebend. Zum einen die Fähigkeit, die Common Query direkt umsetzen zu können, und ein leichter Einbau in die Implementierung.

Common-Query-Konstrukte und Entsprechungen in XSLT/XQuery			
Konstrukt	**Common Query**	**XSLT**	**XQuery**
term ✔	`dc:creator='Knuth'`	mit XPath-Ausdruck: `(dc:creator='Knuth' or @dc:creator='Knuth')`	mit XPath-Ausdruck: `(dc:creator='Knuth' or @dc:creator='Knuth')`
relation ✔	`eq, lt, gt, leq, geq,neq`	mit XPath-Operatoren: `=, <, >, <=, >=, !=`	mit XPath-Operatoren: `=, <, >, <=, >=, !=`
Bedingte Ausführung ✔	*Knoten erfüllt Term*	`<xsl:template match="node[term]"> und <xsl:if test="node[term]">`	WHERE-Klausel und IF-THEN Statement
truncation ✔	`truncation=['left', 'right', 'both']`	mit XSLT-Funktionen: `substring(), string-length(),starts-with(), contains()`	mit XQuery-Funktionen: `xf:ends-with,xf:starts-with, xf:contains`
case-sensitive ✔	`case-sensitive='yes'`	mit XSL- Funktion: `translate()` und Hilfsfunktion für Sonderzeichen (ß->SS)	mit XQuery-Funktion: `xf:lower-case`
boolean ✔	`(Term OP Term)`	mit XPath-Ausdruck: `[(Term) OP (Term)]`	mit XPath-Ausdruck: `[(Term) OP (Term)]`
boolean operator ✔	`op=['and', 'or', 'and-not']`	mit XPath-Boolean: `and, or, (Term and not(Term))`	mit XPath-Boolean: `and, or, (Term and not(Term))`
query ✔	`(Term OP (Boolean))`	mit XPath-Klammerung '()'	mit XPath-Klammerung '()'

[Abbildung 17: Tabelle: Common-Query-Konstrukte und Entsprechungen in XSLT/XQuery]

Der Vergleich der Common Query Konstrukte mit ihren Gegenstücken in XSLT und XQuery zeigt, dass sich fast alles auf XPath-Ausdrücke und wenige Standardfunktionen abstützt. Diese existieren, bis auf wenige Ausnahmen, in beiden Sprachen.
Die Unkompliziertheit und der Umfang an Funktionen ließ daher XQuery zuerst als die geeignetere Wahl erscheinen. Allerdings entschied die Suche nach passenden Implementierungen von Prozessoren die Wahl schnell zugunsten von XSLT.

Bei der Suche waren folgende Kriterien wichtig:

- Ausgereiftheit und Stabilität (Final Release oder späte Beta-Version)
- breite Akzeptanz und weite Verbreitung (garantiert dauerhafter Support, Dokumentation)
- Umsetzung des Standard ohne proprietäre Erweiterungen
- nicht kommerziell (kosten- und lizenzfrei)
- als API-Bibliothek - möglichst „open-source“ - direkt im Code verwendbar

Es fand sich keine XQuery-Umsetzung, die alle diese Kriterien erfüllt.

Mit *Kawa [15]* und *Kweelt [16]* liegen zwar stabile OpenSource-APIs vor, die allerdings eigene Varianten des XQuery Konzeptes oder seiner Vorgänger XQL und XML Query verwenden. Da beide vor dem W3C XQuery Draft entwickelt wurden, erscheint eine Weiterentwicklung und Support fraglich.
Weder als besonders ausgereift noch als dauerhaftes Projekt ausgelegt erschien auch FATDOGs *XML Query Engine [17]*, ein stand-alone XQuery Prozessor. Dieser kann zwar über eine Schnittstelle von Javaprogrammen aufgerufen, aber nicht direkt integriert werden.
Eine Sourceforge Projekt namens *XQuench [18]* erfüllte zwar die meisten der obigen Forderungen, befand sich aber noch in der frühen Konzeptphase.

Der Mangel an geeigneten Umsetzungen erscheint vor dem Hintergrund der aktuellen Diskussion über die Zukunft von XML-Querysprachen und der Neuheit des XQuery-Standards nicht überraschend.
Aus diesem Grund wurde entschieden, als Querysprache XSLT und den bekannten Xalan-J-Prozessor zu verwenden. Dieser ist Teil des bekannten und akzeptierten Apache-Jakarta-Projekts und unterstützt alle aktuellen XML-APIs für Java (z.B. JAXP, DOM, SAX, JDOM).
Wie die Xalan-J-API in den XML-Wrapper eingebunden ist, wird in Kapitel VI beschrieben.

4 Umsetzung von Common Queries in XSLT

Das Konvertieren der Common Query in ein XSLT-Stylesheet übernehmen in der Implementation mehrere Javaklassen. Diese benötigen zum Erstellen die folgenden Quell- und Queryspezifischen Daten:

- eine interne Darstellung der Common Query
- eine Mappingtabelle für die verwendeten Feldbezeichner
- den Elementnamen der Informationseinheit

Die Hauptschritte, die zum Aufbau des XSLT-Codes durchgeführt werden, sollen anhand des Beispiels aus Kapitel III beschrieben werden. Als Datenquelle und Konfigurationsdaten verwenden wir den dort gezeigten Ausschnitt der ACM SigmodRecords, die Konfigurations-datei und die Query (siehe dazu Abb. 6, 9, 3 auf den Seiten 16, 20, 13).
Ohne Zusätze wie `truncation`, lässt sich die Query konzeptuell so darstellen:

```
(dc:creator='C. J. Date' AND dc:identifier>='2000')
AND-NOT (dc:title='database')
```

Zur Erinnerung: *Elemente vom Typ* `article` *wurden als Informationseinheit definiert.* `dc:identifier` *mappt in der Datenquelle auf die Eigenschaft „*`volume`*“. Da diese außerhalb eines* `article`*-Knotens liegt, ist in der Konfigurationsdatei ihr Attribut* `outer` *auf* `true` *gesetzt.*

4.1 Eventuelle Änderung der Struktur (Flattening)

Im einem vorgeschalteten Schritt werden die in Kapitel IV beschriebenen Strukturprobleme abgefangen. Ein zusätzliches XSLT-Stylesheet baut dann intern eine strukturell angepasste Kopie der Datenquelle auf, auf die die eigentliche Query angewendet wird. Das folgende Codebeispiel zeigt ein solches Stylesheet. Die einzelnen Bestandteile werden nachfolgend gesondert erklärt.

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:transform xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="xml" />
    <xsl:template match="node() | @*">
      <flatsource>
         <xsl:apply-templates />
      </flatsource>
    </xsl:template>
    <xsl:template match="article">
      <xsl:copy>
         <xsl:if test="ancestor-or-self::node()[@volume]">
            <xsl:attribute name="volume">
               <xsl:value-of select="ancestor-or-self::node()/@volume" />
            </xsl:attribute>
         </xsl:if>
         <xsl:copy-of select="ancestor::node()/volume" />
         <xsl:copy-of select="*" />
      </xsl:copy>
    </xsl:template>
    <xsl:template match="text()" />
</xsl:transform>
```

[Abbildung 18: XSLT-Stylesheet zur Strukturanpassung]

```
    <xsl:template match="node() | @*">
      <flatsource>
         <xsl:apply-templates />
      </flatsource>
    </xsl:template>
```

Anhand dieses Templates wird die Wurzel des temporären Dokuments (das Element `<flatsource>`) erzeugt und mit `<xsl:apply-templates>` der rekursive Baumdurchlauf

gestartet. Dabei werden solange die Prozessor-eigenen Templates verwendet, bis der aktuelle Knoten auf das zweite Template matcht.

```
<xsl:template match="article">
  <xsl:copy>
    <!-- Attribute im Supertree -> Attribute von article-->
    <xsl:if test="ancestor-or-self::node()[@volume]">
      <xsl:attribute name="volume">
        <xsl:value-of select="ancestor-or-self::node()/@volume" />
      </xsl:attribute>
    </xsl:if>
    <!-- Elemente im Supertree -> Kindknoten von article -->
    <xsl:copy-of select="ancestor::node()/volume" />
    <!-- Kopieren aller Kinder und Attribute von article -->
    <xsl:copy-of select="*" />
  </xsl:copy>
</xsl:template>
```

Ist der aktuelle passierte Knoten ein Element der Informationseinheit `<article>` wird dieses mit `<xsl:copy>` übernommen. Da dieses XSLT-Tag im Gegensatz zu `<xsl:copy-of>` nur das Element selbst ohne Attribute und Kindelemente übernimmt, werden diese weiter unten gesondert mit `<xsl:copy-of select="*" />` kopiert.

Mit `<xsl:copy-of select="ancestor::node()/volume">` wird ein Vorgängerknoten mit dem Namen `volume` in den Baum unterhalb von `<article>` kopiert, falls ein solcher existiert.
Für den Fall, dass die Eigenschaft kein Element sondern ein Attribut eines Vorgängerknotens (z.B. von `<issue>`) ist, wird mit Hilfe des Attribut-Konstruktors `<xsl:attribute>` und `<xsl:value-of>` wird dem noch leeren `<article>`-Element ein neues Attribut `volume` hinzugefügt. Dies ist aber nur dann sinnvoll, wenn ein solches Attribut überhaupt existiert.

Das Ergebnis einer solchen Strukturänderung zeigt Abbildung 7 auf Seite 22

4.2 Aufbau der XPath-Suchbedingung

Nach dem Test der Common Query auf syntaktische Korrektheit, wird sie mit Hilfe der Informationen innerhalb des `<properties>`-Elements in der Konfigurationsdatei Stück für Stück umgeschrieben.

Dabei werden zuerst die generischen Feldbezeichner durch die Element- und Attributnamen ersetzt, die tatsächlich im SigmodRecord-Archiv vorkommen. Sonderbehandlungen wie Trunkierung oder Case-Insensitiveness werden durch XSLT-Funktionen behandelt, und die Syntax wird der von XPath angepasst. Das Ergebnis dieses Vorgangs ist eine String-

Variable, die später direkt in ein vorgefertigtes parametrisierbares XSLT-Gerüst eingefügt wird.
Die folgende Schritte zeigen, wie die Query aus Abbildung 5 umgebaut wird:

Konvertieren der Feldbezeichner:

```
(author='C. J. Date' AND volume>='2000') AND-NOT (title='database')
```

Gesonderte Abfrage für Elemente UND Attribute:

```
(author='C. J. Date' or @author='C. J. Date') AND
(volume>='2000' or @volume>='2000') AND-NOT
(title='database' or @title='database')
```

Ersetzten der booleschen Operatoren durch XPath-Entsprechungen:

```
(author='C. J. Date' or @author='C. J. Date') and
(volume>='2000' or @volume>='2000') and
not(title='database' or @title='database')
```

Anwenden von XSLT-Funktionen für Truncation und Whitespace-Stripping:

```
(normalize-space(.//author)='C. J. Date' or .//@author='C. J. Date) and
(normalize-space(.//volume)>='2000' or .//@volume>='2000') and
not(.//title=[contains(.,'database')] or .//@title[contains(.,'database')])
```

Mit `normalize-space()` werden überflüssige Leerzeichen (z.B. am Ende oder Anfang) einer Zeichenkette gelöscht. Die Funktion `contains()` überprüft ob innerhalb einer Zeichenkette ein Teilstring vorkommt. Sie setzt damit das Attribut `truncation=both` um. Welche anderen Funktionen innerhalb der Implementierung verwendet werden (z.B. bei `case-sensitive=“no“`), wird in Kapitel VI.4 erklärt. Die dort beschriebene Klasse `Term.java` ist nämlich für die Umsetzung in einen XPath-Ausdruck verantwortlich.

Zusatz: Sollte bei der Verwendung von any das mapto-Attribut mehrwertig sein werden die einzelnen Bezeichner mit einem JAVA-StreamTokenizer extrahiert und getrennt behandelt.

```
        ...
        while(tokenizer.hasMoreTokens())
        {
           mappedField = tokenizer.nextToken().trim();
           sofar +=
           ".//" + mappedField + condition + " or .//@" + mappedField + condition;

           if(tokenizer.hasMoreTokens())  sofar += " or ";
        }
        return "(" + sofar + ")";
}
```

4.3 Aufbau des XSLT-Stylesheets

Dieser XPath-Ausdruck wird zusammen mit dem Namen der Informationseinheit in ein XSLT-Gerüst eingefügt. Dieses ist für jede Common Query bzw. verschiedene Datenquellen gleich und sieht beispielsweise so aus:

```
<xsl:transform xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
   <xsl:output method="xml" />
   <xsl:template match="/">
     <results>
        <xsl:apply-templates select="node() | @*" />
     </results>
   </xsl:template>
   <xsl:template match="article">
      <xsl:if test="self::node()
      [(normalize-space(.//author)='C. J. Date' or .//@author='C. J. Date) and
       (normalize-space(.//volume)>='2000' or .//@volume>='2000') and
not(.//title=[contains(.,'database')] or .//@title[contains(.,'database')])]">
        <xsl:copy-of select="." />
      </xsl:if>
   </xsl:template>
   <xsl:template match="text()" />
</xsl:transform>
```

[Abbildung 19: Parametrisiertes XSLT-Stylesheet zur Queryanwendung]

Es besteht aus folgenden Bestandteilen:

```
   <xsl:template match="/">
     <results>
        <xsl:apply-templates select="node() | @*" />
     </results>
   </xsl:template>
```

Die Ausgabedatei mit den Anfrageergebnissen muss XML-formatiert sein und benötigt daher einen Wurzelknoten.
Sobald der XSLT-Prozessor bei der Abarbeitung des Stylesheets in der XML-Datenquelle auf deren Wurzelknoten (`match="/"`) trifft (hier `<SigmodRecord>`), wird im Ausgabedokument ein Wurzelelement mit dem Namen `<results>` erzeugt. Danach werden mit `<xsl:apply-templates>` alle Templates aufgerufen, die auf Knoten oder Attribute matchen. Sinn dieses Aufrufs ist es, zum einen rekursiv die anderen Templates aufzurufen, um den gesamten Dokument- Baum zu durchsuchen, zum anderen aber das Defaulttemplate, welches Elementtext immer in die Ausgabe überträgt, zu unterdrücken.

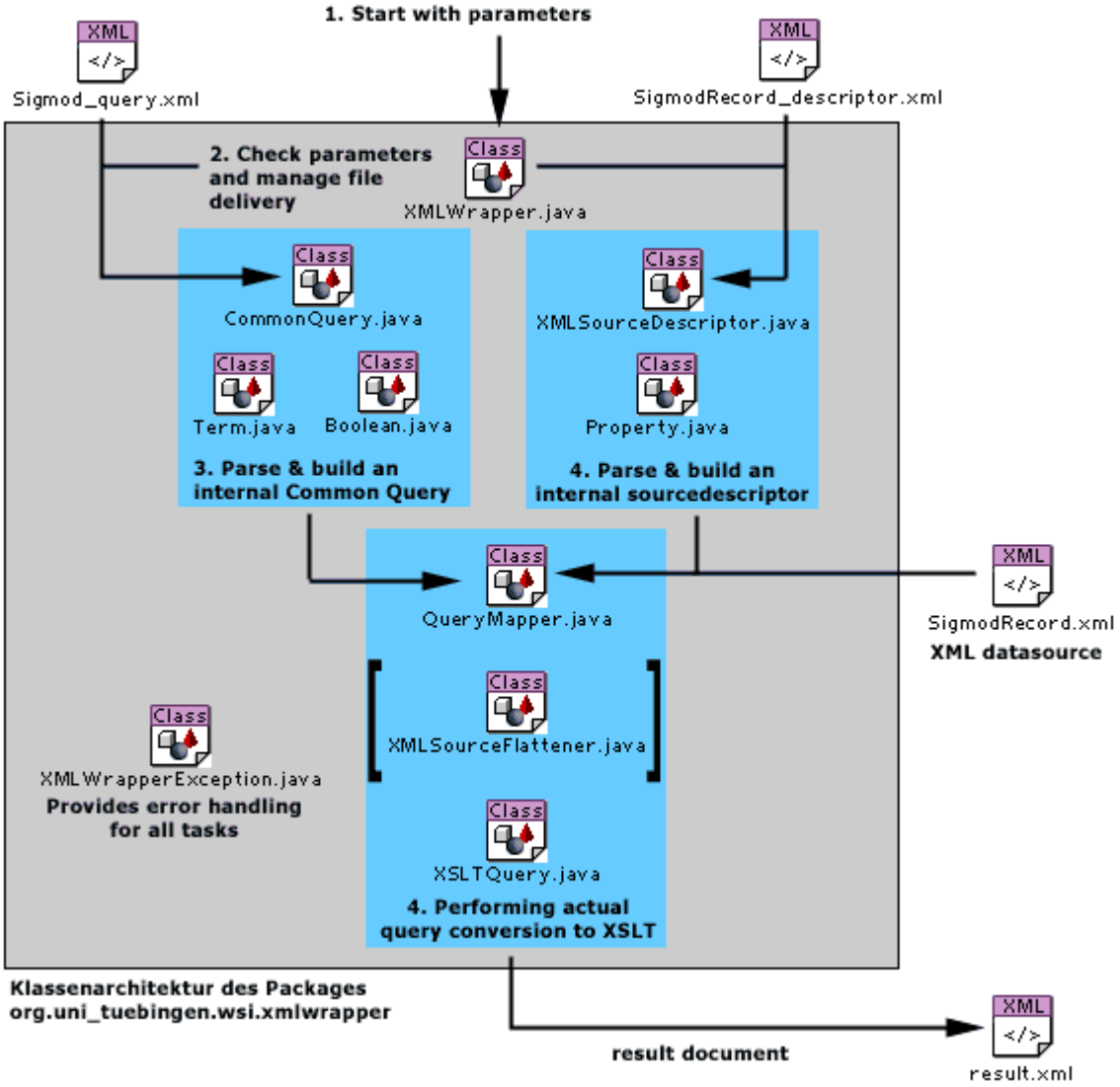
```
<xsl:template match="article">
   <xsl:if test="self::node()[
   (normalize-space(.//author)='C. J. Date' or .//@author='C. J. Date) and
   (normalize-space(.//volume)>='2000') or .//@volume>='2000') and
    not(.//title=[contains(.,'database')] or
                                        .//@title[contains(.,'database')])]">
       <xsl:copy-of select="." />
   </xsl:if>
</xsl:template>
```

Den eigentlichen Kopiervorgang übernimmt das zweite Template. Es matcht auf Elemente, die in der Konfigurationsdatei unter `<infounit>` als Informationseinheit definiert wurden, hier z.B. `<article>`.
Innerhalb der bedingten Anweisung `<xsl:if>` wird geprüft, ob `<article>` und seine Nachfolgerknoten die XPath-Bedingung erfüllen. Dabei wird aus Gründen der Lesbarkeit statt der abgekürzten Schreibweise '.', der XPath-Axis-Specifier `self::node()` verwendet. Ist die Bedingung erfüllt, wird mit `<xsl:copy-of select=“.“>` der gesamte Unterbaum (inkl. `<article>`) in das Resultatdokument übernommen.

Zusätzlich wurde jeder Element- und Attributname in der XPath-Bedingung um die verallgemeinernde Pfadanweisung './/' erweitert.
Ohne diesen Ausdruck würden die Bedingungen nur an direkten Nachfolgern – also Kindknoten – von `<article>` getestet. Da Eigenschaften aber beliebig tief in der Informationseinheit verschachtelt sein können, muss dieser allgemeinere Test auf allen Nachfolger vorgenommen werden.

VI Implementierung und Funktionsweise

Die Implementierung des XML-Wrappers umfasst zehn Java Klassen. Ihre Aufgaben und ihr Zusammenspiel werden in diesem Kapitel kurz beschrieben. Detaillierte Informationen finden sich direkt in den beigelegten Java-Sourcecodes.



[Abbildung 20: Klassenarchitektur der Implementierung]

Alle Klassen wurden innerhalb der OpenSource-IDE *Netbeans* erstellt und getestet. Zur Ausführung des Programms wird wegen der Verwendung einiger Java -2-Features (z.B. Collections Framework) mindestens ein JDK der Version 1.2 vorausgesetzt. Zur Anzeige der Javadoc-Dokumentation wird ein einfacher Webbrowser benötigt.

1 XMLWrapper.java

Die Hauptaufgabe dieser Klasse ist es, die Funktion der anderen Komponenten zu steuern und Hilfsfunktionen für diese bereitzustellen.

Mit dem Aufruf der `main()`-Methode der Klasse `XMLWrapper.java` wird die Implementierung gestartet. Diese Klasse kapselt daher für die Außenwelt die gesamte Funktionalität des XML-Wrapper Packages.
Die Klasse fungiert als nicht-graphisches Stand-alone-Programm, das mit dem Information Broker über Eingabeparameter und Fehlercodes kommuniziert.

Als Parameter erwartet die Klasse:

1. descriptorURL : URL, die auf die Konfigurationsdatei verweist
2. sourceAlias : Referenzname der gewünschten Datenquelle
3. queryURL : URL, die auf die Datei mit der Common Query verweist
4. outputURL (optional) : URL, an die das Ergebnis der Query geschrieben werden soll (bei Fehlen dieses Parameters wird das Ergebnis direkt am Bildschirm angezeigt)

Die URLs können sowohl lokale als auch entfernte Dateien über HTTP und FTP adressieren. Je nach Protokoll werden zusätzliche URL-Parameter benötigt.

Beispiele für URLs:

- file:/c:/studium/wrapper/testsource.xml : Lesen und Schreiben einer lokalen Datei
- http://www.xml.org/test.xml : Lesen und Schreiben (mit HTTP-PUT) von/auf frei zugänglichem HTTP-Server
- http://user1:pass@www.xml.org/secret.xml : Lesen und Schreiben (mit HTTP-PUT) von/auf geschütztem HTTP-Server
- ftp://xml.org/secret.xml : Lesen und Schreiben von/auf frei zugänglichen FTP-Server
- ftp://user1:pass@xml.org/secret.xml:80 : Lesen und Schreiben von/auf geschützten FTP-Server und eines anderen Ports

Die Klasse XMLWrapper erledigt konkret die folgenden Aufgaben:

1. überprüft Parameteranzahl und -syntax (z.B. bei URLs)
2. baut Leseverbindung zu URLs auf und übergibt Java-Inputstreams an Komponenten
3. produziert, empfängt und gibt Fehlermeldungen aus (z.B. in Log-Datei)
4. instanziiert und koordiniert die anderen Komponenten
5. schreibt Queryergebnis auf URL oder gibt es am Bildschirm aus

Die Verbindung zu Quellen im Netz wird durch zwei Fremd-APIs durchgeführt. Für das Handling von FTP-Verbindungen wird die freie `NetComponents API` von Daniel F. Savarese (www.savarese.org) verwendet.

Obwohl das `java.net` Package des JDK Zugriff auf HTTP-Quellen erlaubt, unterstützt es keinerlei Schreibzugriffe per HTTP-PUT. Für diese Aufgabe verwendet diese Klasse daher die freie `HTTPClient`-API der Schweizer Firma innovation.ch.
Treten bei der Verwendung dieser oder einer anderen Klasse Fehler oder Exceptions auf, werden diese abgefangen und in einer Logdatei mitprotokolliert. Dort werden die genaue Fehlerzeit und die benutzten Eingabeparameter vermerkt.

Der Zugriff auf die Inhalten der XML-Dateien wird den anderen Klassen über Streams oder durch interne XML-Repräsentationen ermöglicht.
Dabei wird die freie JDOM-API (www.jdom.org) verwendet *(siehe dazu auch [8]).*
Das interne XML-Baumobjekt das JDOM im Speicher aufbaut, lässt sich eleganter und einfacher manipulieren als das von DOM- oder SAX-Parsern. Es lassen sich auf einfachste Weise XML-Dokumente einlesen oder intern generieren und anschließend als Datei ausgeben. JDOM wird extensiv von fast allen Klassen des Packages verwendet.

2 XMLWrapperException.java

Die Hauptaufgabe dieser Klasse ist die Behandlung von Exceptions und die Ausgabe aussagekräftiger Fehlermeldungen in der Log-Datei oder an der Kommandozeile.

Es gibt zwei verschiedene Fehlerquellen. Entweder erzeugt der XML-Wrapper selbst Fehler (z.B. falsche Syntax der Eingabeparameter/ -dokumente) oder eine der verwendeten Fremdkomponenten.
Die Funktionsweise dieser Komponenten ist für den Nutzer versteckt und ihre Fehlermeldungen sind daher oft unverständlich, da sie sich nicht in den Funktionskontext des XML-Wrappers einordnen lassen.

Aus diesem Grund kapselt eine gesonderte Klasse interne und externe Fehlermeldungen. Diese werden bei ihrem Auftreten nicht sofort behandelt, sondern als Instanzen der Klasse `XMLWrapperException` gekapselt und so lange in der Aufrufhierarchie nach oben weitergereicht bis sie aussagekräftig beschrieben werden können.
Produziert z.B. die FTP-Komponente einen Zugriffsfehler, kann erst durch dieses Weiterreichen festgestellt werden, dass dies z.B. beim Zugriff auf die Common Query aufgetreten ist.
Außerdem bietet die Klasse Methoden, um den gesamten Fehlerpfad (=stack trace) auszugeben, was das Debuggen sehr erleichtert.

3 XMLSourceDescriptor.java und Property.java

Die Hauptaufgabe dieser beiden Klassen ist das Parsen von Konfigurationsdateien und das Bereitstellen und Aufbereiten ihrer Informationen für die Klasse QueryMapper.java.

Konfigurationsdateien werden eingelesen, in JDOM-Objekte überführt und auf korrekte Syntax hin überprüft. Dann wird ein internes Abbild der darin enthaltenen Informationen gespeichert. So kann z.B. später durch Aufruf der Selektormethode `getSourceURL()` eine

andere Klasse die Adresse der eigentlichen XML-Datenquelle erfahren, um diese herunterzuladen.
Aus Gründen der Übersichtlichkeit werden die `<property>`-Elemente aus Konfigurationsdateien von der separaten Klasse `Property.java` verarbeitet. Alle Eigenschaften, wie z.B. die Dublin-Core Namen und die konkreten Elementnamen werden intern als `HashMap` gespeichert.

4 CommonQuery.java, Term.java und Boolean.java

Die Hauptaufgabe dieser drei Klassen ist das Parsen von Common Queries und das Aufbereiten und Bereitstellen ihrer Informationen für die Klasse QueryMapper.java.

Analog zu Konfigurationsdateien werden Common Queries behandelt. Je nach deren innerem Aufbau kapselt ein Objekt der Klasse `CommonQuery` mehrere Unterobjekte vom Typ `Boolean` und `Term`. Diese Klasse überprüft nur die korrekte Syntax des Wurzelknotens `<query>` und die korrekte Anzahl von Kindknoten. Findet sie dabei Elemente vom Typ `<term>` oder `<boolean>`, erledigen rekursiv die Instanzen der entsprechenden Klassen das Parsen.

Allen drei Klassen ist eine wichtige Methode gemeinsam. Die Methode `toXSLTPattern()` konvertiert das jeweilige Common-Query-Konstrukt in einen gültigen XPath-Teilausdruck. Wie der Codeausschnitt zeigt, ruft diese Methode in einem `CommonQuery`-Objekt die gleichnamige Methode der Unterobjekte von Typ `Term` oder `Boolean` auf und baut aus den Ergebnissen die in Abschnitt 4.2 von Kapitel V beschriebene XPath-Suchbedingung auf.

```
public String toXSLTPattern(String sofar, HashMap sourceProperties)
{
   if(termOrBoolean instanceof Term)
     return ((Term) termOrBoolean).toXSLTPattern(sofar, sourceProperties);
   else
     return ((Boolean) termOrBoolean).toXSLTPattern(sofar,sourceProperties);
}
```

[Abbildung 21: Auszug CommonQuery.java]

Während hier nur entschieden wird, welcher Konstruktor aufzurufen ist, finden in den Methoden der anderen zwei Klassen die eigentlichen Ersetzungen statt.

```
public String toXSLTPattern(String sofar , HashMap sourceProperties)
{
   String firstPattern;
   String secondPattern;

   //<boolean> can be all binary permutations of terms and other booleans
   if(first instanceof Term)
     firstPattern = ((Term) first).toXSLTPattern("", sourceProperties);
```

```
    else
      firstPattern = ((Boolean) first).toXSLTPattern("", sourceProperties);

    if(second instanceof Term)
      secondPattern = ((Term) second).toXSLTPattern("", sourceProperties);
    else
      secondPattern = ((Boolean) second).toXSLTPattern("", sourceProperties);

    // special conversion needed for 'and-not'
    if(operator.equals("and-not"))
      sofar += "(" + firstPattern + " and not(" + secondPattern + "))";
    else
      sofar += "(" + firstPattern + " " + operator + " " + secondPattern + ")";

    return sofar;
}
```

[Abbildung 22: Auszug Boolean.java – Klammerung + Konvertierung Boolescher Operatoren]

Wegen der Länge der Methode `toXSLTPattern()` in der Klasse `Term.java` werden im folgenden nur die interessantesten Auszüge gezeigt.

```
    ...
    // default: values are case-insensitive
    if(case_sensitive.equals("no"))
    {
       // make actual value and query value both lowercase
       value = "translate(" + value + ", $UPPERCASE, $LOWERCASE)";
       originalvalue = "translate(., $UPPERCASE, $LOWERCASE)";
    }
    ...
```

Beim `value`-Attribut in Termen wird default-mäßig nicht zwischen Groß-und Kleinschreibung unterschieden. Da XSLT über keine echte `toLowerCase()`-Methode verfügt, wird stattdessen die `translate`-Funktion verwendet. Sie ersetzt alle Zeichen eines Strings, die in der Variable UPPERCASE enthalten ist durch die entsprechenden Zeichen in der Variable LOWERCASE.

Beide Variablen werden innerhalb des später verwendeten Stylesheet-Gerüstes definiert und haben die folgende Form: **$UPPERCASE = “ABCDEFGHIJ...“** bzw. **$LOWERCASE = “abcdefghij...“**
Um auch exotische Sprachen konvertieren zu können, hat jede der beiden Strings mehrere tausend Zeichen. Es wird daher verzichtet, sie in der Ausarbeitung in voller Länge zu zeigen.

```
    ...
    // always normalize spaces in and around field
    originalvalue = "normalize-space(" + originalvalue + ")";
    ...
```

Die XSLT-Funktion `normalize-space()` eliminiert überflüssige Leerzeichen in `value`.

```
    ...
    if(truncation.equals("both"))
      condition = "[contains(" + originalvalue + ", " + value + ")]";
    else if(truncation.equals("right"))
      condition = "[starts-with(" + originalvalue + ", " + value + ")]";
    else if(truncation.equals("left"))
      condition = "[substring(" + originalvalue + ", string-length(.) -
                  string-length('" + value + "') + 1)=" + value + "]";
    else
      condition = "[" + originalvalue + " " + relation + " " + value + "]";

    //check element and attribute values seperately
    sofar += "(.//" + mappedField + condition + " or .//@" + mappedField +
                condition + ")";
    return sofar;
}
```

Je nach Wert des `truncation`-Attributes wird eine andere Funktionen benötigt. Da es in XSLT kein `ends-with()` gibt, muss die Funktion mit `substring()` und `string-length()` imitiert werden.
Abschließend wird der oben beschriebene separate Test von Attributen und Elementen umgesetzt.

5 QueryMapper.java

Die Hauptaufgabe dieser Klasse ist es, für eine bestimmte Datenquelle die Umwandlung von Common Queries in ein XSLT-Stylesheet zu regeln.

Die Klasse hat neben dem Konstruktor nur eine weitere Methode. Diese parametrisiert für ein bestimmtes `CommonQuery`-Objekt das in Abschnitt 4.3 des Kapitels V beschriebene Stylesheet-Gerüst. Je nach Bedarf wird vorher ein gesondertes Flattening-Stylesheet aufgerufen, welches das „Ausmultiplizieren“ außerhalb liegender Eigenschaften vornimmt.

Dabei wird der Fall, dass der Feldbezeichner any auf mehrere Namen gemappt wird gesondert behandelt. Mit einem JAVA-`StringTokenizer` werden die durch Kommata getrennten Namen extrahiert und in die ArrayList `outerProperties` eingetragen. Hierbei wird nicht unterschieden, welches der einzelnen Teileigenschaften eigentlich oberhalb der Informationseinheit liegt. Damit werden später bei „Ausmultiplizieren“ alle auf Ihre Lage hin überprüft.

Die nähere Funktionsweise zeigt der folgende Code der Methode `executeQuery()`:

```
public Document executeQuery(CommonQuery commonQuery)
{
    HashMap propertiesMap = descriptor.getPropertiesMap();
    Iterator iterator = propertiesMap.values().iterator();
    ArrayList outerProperties = new ArrayList();
    boolean hasOuters = false;
    StringTokenizer tokenizer;

    // handle outer properties seperately
    while(iterator.hasNext()) {
      Property p = (Property) iterator.next();

      if(p.getOuter().equals("true")) {
           outerProperties.add(new String(p.getMapto()));
           hasOuters = true;

           // if mapTo belongs to complex 'any' fieldname
           if(p.getMapto().indexOf(",")!=-1) {
               tokenizer = new StringTokenizer(p.getMapto(), ",");

               while(tokenizer.hasMoreTokens()) {
                  outerProperties.add(new String(tokenizer.nextToken().trim()));
               }
           }
```

```
            // else if mapTo is single value
      else outerProperties.add(new String(p.getMapto()));
   }

   String condition = commonQuery.toXSLTPattern("", propertiesMap);
   String infoUnit = descriptor.getInfounit();
   String resultdtd_url = descriptor.getResultdtd_url();
   // get actual source XML file
   source = new JDOMSource(builder.build(descriptor.getSource_url()));

   // eventually overwrite source with flattened copy
   if(hasOuters) {
          XMLSourceFlattener flattener =
          new XMLSourceFlattener(source, infoUnit);
          source = new JDOMSource(flattener.flatten(outerProperties));
   }
   // return query result provided by XSLT
   XSLTQuery xsltQuery = new XSLTQuery(source, infoUnit, resultdtd_url);
   return xsltQuery.execute(condition);
}
```

6 XMLSourceFlattener.java

Die Hauptaufgabe dieser Klasse ist es, ein parametrisierbares Stylesheet-Gerüst zu generieren, das sog. „outer“-Eigenschaften „ausmultipliziert“.

Im vorherigen Codefragment wird in Fällen, in denen die Konfigurationsdatei „outer“-Elemente ausweist, eine Instanz dieser Klasse instantiiert.
Der Konstruktor und die Methode `flatten()` erwarten dazu als Parameter das Ausgangsdokument als JDOM-`Document`-Objekt, den Namen der Informationseinheit und die Liste der auszumultiplizierenden Eigenschaften in Form einer einfachen `ArrayList`.
Sollten dabei
Mit Methoden der JDOM-API wird ein XML-Dokument erzeugt, das ein XSLT-Stylesheet analog zum Beispiel aus Abschnitt 4.1 von Kapitel V erzeugt. Die eigentliche Transformation des Ausgangsdokuments übernimmt der XSLT-Prozessor Xalan des Apache-Jakarta-Projekts.

Das folgende Codefragment zeigt, wie die Xalan-API eingebunden und verwendet wird:

```
import org.jdom.transform.*;        // import statements for
import javax.xml.transform.*;       // Xalan-J Processor
...
public Document flatten(ArrayList outerProperties) throws Exception
{
   TransformerFactory transformerFactory = TransformerFactory.newInstance();

   // build stylesheet that performs this flattening
   Document stylesheetFrame = getFlatteningXSLT(outerProperties);

   // create Transformer object for stylesheet
   JDOMSource querySource = new JDOMSource(stylesheetFrame);
   Templates queryTemplate = transformerFactory.newTemplates(querySource);
   Transformer processor = queryTemplate.newTransformer();

   // perform XSLT transformation and return the resulting JDOM document
   JDOMResult result = new JDOMResult();
   processor.transform(source, result);

   return result.getDocument();
}
```

Die Methode `getFlatteningXSLT()` erzeugt für eine Liste von `outer`-Eigenschaften ein Stylesheet, das von einem `Transformer`-Objekt auf die Datenquelle `source` angewendet wird, d.h. diese wird gemäß der Templates im Stylesheet transformiert.

Das Ergebnis dieses Preprocessings wird an den aufrufenden QueryMapper zurückgegeben, der es an ein Objekt der folgenden Klasse weiterreicht. Erst dort findet dann die eigentliche Abarbeitung der Common Query statt.

7 XSLTQuery.java

Die Hauptaufgabe dieser Klasse liegt darin, die in XSLT-Syntax konvertierte Nutzeranfrage an die eigentliche XML-Datenquelle zu stellen und das Ergebnisdokument bereitzustellen.

Im nachfolgenden Codefragment wird gezeigt, wie anhand von Objekten und Methoden der JDOM-API das XSLT-Stylesheet-Gerüst aus Abschnitt 4.3 von Kapitel V aufgebaut wird.

```
private Document getXSLTDocument(String condition){
    // builds <xsl:transform xmlns="http://www.w3.org.." version="1.0">-Element
    Element transform = new Element("transform","xsl","http://www.w3.org/..");
    transform.setAttribute("version", "1.0");
    ...
    // builds <xsl:template match="/">-Element
    Element identity = new Element("template", XMLWrapper.XSLNAMESPACE);
    identity.setAttribute("match", "/");

    // builds output root node <results>
    Element result = new Element("results");
    result.addContent(apply);
    identity.addContent(result);

    // builds <xsl:apply-templates select="node() | @*" />-Element
    Element apply = new Element("apply-templates", XMLWrapper.XSL_NACE);
    apply.setAttribute("select", "node() | @*");
    transform.addContent(identity);
    ...
    // builds <xsl:template match="article">-Element from infoUnit variable
    Element querymatcher = new Element("template", XMLWrapper.XSL_NAMESPACE);
    querymatcher.setAttribute("match", infoUnit);
    transform.addContent(querymatcher);

    // builds <xsl:if test="self::node()[..]">-Element from query condition
    Element infounit_if = new Element("if", XMLWrapper.XSL_NAMESPACE);
    infounit_if.setAttribute("test", "self::node()[" + condition + "]");
    infounitTemplate.addContent(infounit_if);

    // builds <xsl:copy-of select="." />-Element
    Element copy_of = new Element("copy-of", XMLWrapper.XSL_NAMESPACE);
    copy_of.setAttribute("select", ".");
    infounit_if.addContent(copy_of);
    ...
    return new Document(transform);
}
```

Analog zur Klasse `XMLSourceFlattener.java` wird dieses Stylesheet mit dem Xalan-Prozessor umgesetzt. Das Resultat wird abschließend von der Klasse `XMLWrapper.java` entweder an eine Ausgabe-URL versendet oder direkt auf dem Bildschirm ausgegeben.

Einen kompletten Überblick über alle Ein- und Ausgaben, sowie alle Zwischenschritte (in etwa das Eingabedokument nach dem Flattening) einer typischen Anwendung bietet Abschnitt 1 im Anhang.

VII Einschränkungen und Ausblick

Der XML-Wrapper wurde in seiner jetzigen Form anhand mehrerer repräsentativer XML-Datenquellen bzw. Anfragen getestet und erledigte seine Aufgabe erwartungsgemäß erfolgreich. Allerdings ist vieles noch verbesserungswürdig. Im folgenden soll noch knapp auf größere Einschränkungen und eventuelle Erweiterungsmöglichkeiten eingegangen werden.

1 Nachteile von Strukturänderungen

Die Vielfalt der Möglichkeiten, ein und dieselben Daten zu modellieren, macht unter bestimmten Bedingungen eine Vorverarbeitung der Eingabedatei nötig.
Außer dass sich dadurch der Zeitaufwand pro Query ungefähr verdoppelt, ist eine Änderung der Original-Struktur nötig. Das Ausgabedokument folgt in fast allen Fällen nicht mehr der ursprünglichen Grammatik des Eingabedokuments.
Das Vorhandensein einer solchen DTD ist allerdings für die maschinelle Weiterverarbeitung des Ergebnisses wichtig. Nur mit ihrer Hilfe kann ein Prozessor oder Parser die vorhandene Struktur erkennen und das Dokument effizient bearbeiten.
Das automatische Generieren einer neuen DTD anhand weniger bekannter Dokumente ist kompliziert oder gar unmöglich.
Ein möglicher Ansatz wäre, die vorhandene DTD während des Ausmultiplizierens analog mitzutransformieren. Dies wurde allerdings aus Zeitgründen im Rahmen der Studienarbeit nicht weiter verfolgt.

Um der Verdoppelung der Rechenzeit zu begegnen, könnte die XML-Datenquelle nach bereits durchgeführtem Flattening verwendet werden. So müsste sie nur einmal und nicht bei jeder Query immer wieder neu transformiert werden.

2 Einschränkung der Feldbezeichner-Domänen

Eine weitere Einschränkung ist der Umfang der Feldbezeichner. Mit Dublin-Core liegen nur sinnvolle Namen für bibliographische Datenbanken vor.
Zwar ist es jederzeit möglich, in der Konfigurationsdatei diese für jede andere Domäne zu missbrauchen (z.B. `dc:creator` für Preise), aber es ist eher unwahrscheinlich, dass ein Nutzer bei der Preisrecherche in seiner Query `dc:creator` verwenden wird.

Es bleibt daher zu hoffen, dass es bald auch für andere Domänen ähnliche Standardisierungsbemühungen geben wird. Andernfalls müsste für das Information-Broker-System ein eigenes intuitives und kontrolliertes Vokabular entwickelt werden, das auch andere relevante Domänen erfasst.

3 Performancegrenzen

Um XML-Dokumente zu parsen, intern zu speichern und zu verarbeiten, müssen sie bisher noch komplett eingelesen werden. Dies ist bei Dateien im Umfang weniger Megabytes unproblematisch.
Es ist aber zu erwarten, dass in Zukunft der Umfang von XML-Datenbeständen immer größer wird. Dies kann zu unakzeptablen Wartezeiten bei der Umsetzung der Query führen.

Ein Ausweg aus dieser Problematik liegt in der Verwendung anderer APIs. Sowohl die JDOM-API, als auch der verwendete Xalan-Prozessor bauen von einem XML-Dokument intern eine baumartige Kopie auf. Dies ist zwar bei der Verarbeitung sehr intuitiv, aber auch uneffizient, da z.B. bei der Transformierung mit XSLT diese Bäume zeitweise intern oft 10-100x so groß werden.
Eventbasierte Parser und Prozessoren à la SAX und SAXON, können auch auf unvollständig eingelesenen Daten agieren. Da sie in ihrem Handling wesentlich komplizierter und ihren Möglichkeiten eingeschränkter sind als DOM-Tools, wurden sie in der aktuellen Implementation nicht verwendet.

Ebenfalls sollte über die Verwendung anderer APIs für die Zugriffe auf entfernte FTP- und HTTP-Server nachgedacht werden. Sowohl aus der Savarese Netcomponents-API, als auch dem HTTPClient werden nur wenige Klassen verwendet. Diese sollten - wenn möglich - entweder durch Komponenten des JDKs oder anderer ausgereifterer Standard-APIs ersetzt werden.

4 Ausblick

Mit dem Projekt INFORMATION BROKER von Andreas Ludwig ensteht ein praktisches Tool, das helfen kann, einem der großen Probleme der vielzitierten Informationsgesellschaft zu begegnen - dem berühmten Information-Overkill.

Je öfter klassische Suchmaschinen an ihre Grenzen stoßen und die Internetrecherche zur Farce wird, desto mehr muss über neue Wege der Informationsbeschaffung nachgedacht werden.
Verbesserte Suchalgorithmen wie Googles PageRank-Verfahren sind zwar erfolgreich, gehen aber nicht weit genug, wobei auf der anderen Seite hochgegriffene Pläne eines semantischen Webs, wie es Tim Berners-Lee propagiert, eher illusorisch anmuten.

Der Information Broker ist ein ausgewogener Mittelweg. Er schafft mit seinem erweiterten Querykonzept den nötigen Schritt aus der Schlüsselwort-Misere und fordert dabei trotzdem keine Revolution der existierenden Datenquellen.
Ausserdem macht die offene Komponentenarchitektur es jederzeit möglich, Wrapper für heute noch gänzlich unbekannte Datenquellen hinzuzufügen. Mit minimalem menschlichem Beschreibungsaufwand ist es der hier entwickelten XML-Wrapperkomponente möglich, eine riesige Menge potentieller Datenquellen erreichbar zu machen.
Dass es sich dabei um XML, das wohl erfolgversprechendste Datenformat der näheren Zukunft handelt, macht den Information Broker erst recht zu einem Projekt mit Zukunft.

VIII Anhang

1 Ein- / Ausgaben und Zwischenschritte – Ein Szenario

Der folgende Abschnitt zeigt noch einmal komplett alle Eingaben, Zwischenschritte und Ausgaben eines beispielhaften Nutzungsszenarios.

Start des Programms durch Aufruf der beigelegten Batchdatei:

```
c:\implementierung\bin> XMLWrapper.bat
     file:/d:/implementierung/examples/SigmodRecord_descriptor.xml
     ACMSigmodRecord
     file:/d:/implementierung/examples/Sigmodquery.xml
     file:/d:/implementierung/examples/output.xml
```

Inhalt Konfigurationsdatei SigmodRecord_descriptor.xml:

```
<XMLSourceDescriptor>
   <source-url>
http://www.acm.org/sigs/sigmod/record/xml/Record/SigmodRecord/SigmodRecord.xml
   </source-url>
   <infounit>article</infounit>
   <resultdtd-url>
      file:/d:/studienarbeit/dtd/SigmodRecord_result.dtd
   </resultdtd-url>
   <properties>
      <property field="dc:creator" mapto="author" />
      <property field="dc:type" mapto="position" />
      <property field="dc:date" mapto="number" outer="true" />
      <property field="dc:identifier" mapto="volume" outer="true" />
      <property field="any" mapto="title" outer="true" />
   </properties>
</XMLSourceDescriptor>
```

Inhalt der Common Query SigmodRecordquery.xml:

```
<query>
  <boolean op="and-not">
            <boolean op="or">
                <term value="Date"
                       field="dc:creator"
                       relation="eq"
                       truncation="both" />
                <term value="2000"
                       field="dc:date"
                       relation="geq" />
            </boolean>
            <term value="Database"
                   field="dc:title"
                   relation="eq"
                   truncation="both"
                   case-sensitive="yes"/>
          </boolean>
</query>
```

Ausschnitt der Datenquelle SigmodRecord.xml:

```
<SigmodRecord>
   <issue>
     <volume>1999/volume>
     <number>1</number>
     <articles>
        <article>
           <title>Remarks on Two New Theorems of Date and Fagin</title>
           <initPage>57</initPage>
           <endPage>58</endPage>
           <authors>
              <author position="00">C. J. Date</author>
              <author position="01">Ronald Fagin</author>
           </authors>
        </article>
        <article>
           <title>Architecture of Future Database Systems.</title>
           <initPage>30</initPage>
           <endPage>44</endPage>
           <authors>
              <author position="00">Lawrence A. Rowe</author>
              <author position="01">Michael Stonebraker</author>
```

```
            </authors>
         </article>
      ... weitere Artikel ...
      </articles>
   </issue>
   ...
   <issue>
      <volume>2001</volume>
      <number>3</number>
      <articles>
         <article>
            <title>A Formal Definition of the relational Model.</title>
            <initPage>18</initPage>
            <endPage>29</endPage>
            <authors>
               <author position="00">C. J. Date</author>
            </authors>
         </article>
         <article>
            <title>Database Activity at Aberdeen University.</title>
            <initPage>6</initPage>
            <endPage>6</endPage>
            <authors>
               <author position="00">Stephen Knowles</author>
            </authors>
         </article>
      </articles>
   </issue>
   ...weitere Issues...
</SigmodRecord>
```

Inhalt des XSLT-Stylesheets zum Ausmultiplizieren von <volume>:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:transform xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="xml" />
    <xsl:template match="node() | @*">
      <flatsource>
        <xsl:apply-templates />
      </flatsource>
    </xsl:template>
    <xsl:template match="article">
      <xsl:copy>
        <xsl:if test="ancestor-or-self::node()[@volume]">
          <xsl:attribute name="volume">
             <xsl:value-of select="ancestor-or-self::node()/@volume" />
          </xsl:attribute>
        </xsl:if>
        <xsl:if test="ancestor-or-self::node()[@number]">
          <xsl:attribute name="number">
             <xsl:value-of select="ancestor-or-self::node()/@number" />
          </xsl:attribute>
        </xsl:if>
        <xsl:copy-of select="ancestor::node()/volume" />
        <xsl:copy-of select="ancestor::node()/number" />
        <xsl:copy-of select="*" />
      </xsl:copy>
    </xsl:template>
    <xsl:template match="text()" />
</xsl:transform>
```

Inhalt der Quelle nach dem Ausmultiplizieren:

```
<flatsource>
        <article>
          <volume>1999/volume>
          <number>1</number>
          <title>Remarks on Two New Theorems of Date and Fagin</title>
          <initPage>57</initPage>
          <endPage>58</endPage>
          <authors>
             <author position="00">C. J. Date</author>
             <author position="01">Ronald Fagin</author>
          </authors>
        </article>
```

```
        <article>
          <volume>1999/volume>
          <number>1</number>
          <title>Architecture of Future Data Base Systems.</title>
          <initPage>30</initPage>
          <endPage>44</endPage>
          <authors>
            <author position="00">Lawrence A. Rowe</author>
            <author position="01">Michael Stonebraker</author>
          </authors>
        </article>
     ... weitere Artikel ...
        <article>
          <volume>2001</volume>
          <number>3</number>
          <title>A Formal Definition of the relational Model.</title>
          <initPage>18</initPage>
          <endPage>29</endPage>
          <authors>
            <author position="00">C. J. Date</author>
          </authors>
        </article>
        <article>
          <volume>2001</volume>
          <number>3</number>
          <title>Database Activity at Aberdeen University.</title>
          <initPage>6</initPage>
          <endPage>6</endPage>
          <authors>
            <author position="00">Stephen Knowles</author>
          </authors>
        </article>
<flatsource>
```

Inhalt des XSLT-Stylesheets mit der Nutzer-Anfrage:

```
<xsl:transform xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
   <xsl:output method="xml" />
   <xsl:variable name=“UPPERCASE“ select=“ABCDEFGH...
   <xsl:variable name=“LOWERCASE“ select=“abcdefgh...
   <xsl:template match="/">
     <results>
        <xsl:apply-templates select="node() | @*" />
     </results>
   </xsl:template>
   <xsl:template match="article">
      <xsl:if test="self::node()
      ["self::node()[((((.//author[contains(normalize-space(translate(.,
$UPPERCASE, $LOWERCASE)), translate('Date', $UPPERCASE, $LOWERCASE))] or
.//@author[contains(normalize-space(translate(., $UPPERCASE, $LOWERCASE)),
translate('Date', $UPPERCASE, $LOWERCASE))])) and (.//volume[normalize-
space(translate(., $UPPERCASE, $LOWERCASE))  >=  translate('2000', $UPPERCASE,
$LOWERCASE)] or .//@volume[normalize-space(translate(., $UPPERCASE,
$LOWERCASE))  >=  translate('2000', $UPPERCASE, $LOWERCASE)])) and not
((.//title[contains(normalize-space(.), 'Database')] or
  .//@title[contains(normalize-space(.), 'Database')]))))]">
        <xsl:copy-of select="." />
      </xsl:if>
   </xsl:template>
   <xsl:template match="text()" />
</xsl:transform>
```

Ausgabedokument:

```
<results>
        <article>
           <volume>2001</volume>
           <number>3</number>
           <title>A Formal Definition of the relational Model.</title>
           <initPage>18</initPage>
           <endPage>29</endPage>
           <authors>
              <author position="00">C. J. Date</author>
           </authors>
        </article>
</results>
```

2 Literaturverzeichnis / Ressourcen

Broker / Wrapper / Mediatoren allgemein:

[1] *Jianguo Lu and John Mylopoulos:* Extensible Information Brokers, URL: http://www.cs.toronto.edu/~jglu/pub/xib.pdf

[2] *I. Manolescu, D. Florescu, D. Kossmann:* Answering XML Queries over Heterogeneous Data Sources, URL: http://www.cs.uu.nl/docs/vakken/gd/answering_xml_queries.pdf

[3] *George A. Mihaila, L. Raschid:* Locating Data repositories Using XML, URL: http://www.w3.org/TandS/QL/QL98/pp/ws.html

XML:

[4] XML – Extensible Markup Language, URL: http://www.w3.org

[5] XML.org – The XML Industry Portal, URL: http://www.xml.org

[6] XML.com, URL: http://www.xml.com

JDOM / DOM:

[7] JDOM.org, URL: http://www.jdom.org

[8] *Elliotte Rusty Harold*: JDOM Presentation, URL: http://www.ibiblio.org/xml/slides/xmlsig/jdom

XQuery / XML-Query- Prozessoren:

[9] XQuery 1.0: An XML Query Language, URL: http://www.w3.org/TR/xquery

[10] *Howard Katz:* An introduction to Xquery, URL: http://www-106.ibm.com/developerworks/xml/library/x-xquery.html

[11] *Bas de Bakker und Irsan Widato*: An Introduction to Xquery, URL: http://www.perfectxml.com/articles/xml/xquery.asp

[12] *Leigh Dodds:* Does XML Query Reinvent the wheel, URL: http://www.xml.com/2001/02/28/deviant.html

[13] *Leigh Dodds:* XQuery Questioned, URL: http://www.xml.com/2002/01/02/xquery.html

[14] *Evan Lenz:* Xquery: Reinventing the Wheel?, URL: http://xmlportfolio.com/xquery.html

[15] KAWA XQuery: Prozessor-Implementation URL: http://www.gnu.org/software/kawa/xquery

[16] Kweelt XML Query Prozessor URL: http://kweelt.sourceforge.net

[17] Sourceforge XQuench: XQuery Java API URL: http://sourceforge.net/projects/xquench

[18] Fatdog Software: XML Query Engine
URL: http://www.fatdog.com/#XQUERY

XSLT:

[19] XSLT – XSL Transformations, URL: http://www.w3.org/TR/xslt

[20] TransQuery – XSLT Transformations as Query Language, URL: http://transquery.org

[21] *Dave Pawson:* Things XSLT can't do,
URL: http://www.dpawson.co.uk/xsl/sect2/nono.html

[22] Xalan-Java Version 2.3.1, URL: http://xml.apache.org/xalan-j

[23] Sample Chapters from the XML Bible – XSL Transformations
URL: http://www.ibiblio.org/xml/books/bible2/chapters/ch17.html

[24] XSLT and Xpath Quick Reference,
URL: http://www.mulberrytech.com/quickref/XSLTquickref.pdf

[25] XML Syntax Quick Reference,
URL: http://www.mulberrytech.com/quickref/XMLquickref.pdf

[26] Mulberrytech XSL List (Beiträge/Antworten aus Mailingliste),
URL: http://www.mulberrytech.com/xsl/xsl-list/index.html

Testdatenquellen

[27] XML Query Use Cases, URL: http://www.w3.org/TR/xmlquery-use-cases

[28] ACM SigmodRecord: XML Version, URL: http://www.acm.org/sigs/sigmod/record/xml/Record/SigmodRecord/SigmodRecord.xml

[29] Sample MEDLINE records in XML,
URL: http://www.nlm.nih.gov/databases/dtd/medline_sample.xml

[30] The Plays of Shakespeare in XML,
URL: http://www.oasis-open.org/cover/bosakShakespeare2000.html

Anderes:

[31] Dublin Core Qualifiers, URL: http://www.dublincore.org/documents/dcmes-qualifiers

[32] DC Metadata Element Set, URL: http://www.dublincore.org/documents/dces

[33] *Eric van der Vlist:* Using W3C XMLSchema,
URL: http://xml.com/pub/a/2000/11/29/schemas/part1.html

3 Abbildungsverzeichnis

Abb. [1] Der Information Broker und seine Komponenten 10
Abb. [2] DTD der Common Query Markup Language 14
Abb. [3] Common Query Beispiel 1 15
Abb. [4] Common Query Beispiel 2 mit Boolesche verknüpften Termen 16
Abb. [5] Common Query Beispiel 3 mit Vergleichsrelationen 17
Abb. [6] Ausschnitt XML-Datenquelle SigmodRecord.xml 17
Abb. [7] „Ausmultiplizieren“ übergeordneter Eigenschaften 22
Abb. [8] DTD für Konfigurationsdateien 22
Abb. [9] Beispiel-Konfigurationsdatei für SigmodRecord.xml 23
Abb. [10] XML-Ausgangsdokument zur Erläuterung von XSLT 25
Abb. [11] XSLT Beispiel 1: Konversion XML -> HTML 25
Abb. [12] XSLT Beispiel 2: Konversion XML -> XML 26
Abb. [13] XSLT Beispiel 3: Bedingte Anweisung mit xsl:if 27
Abb. [14] XQuery Beispiel 1: Konversion XML -> HTML 29
Abb. [15] XQuery-Beispiel 2: FLWR-Expression 30
Abb. [16] XQuery-Beispiel 3: Bedingte Auswertung mit IF-THEN-ELSE 30
Abb. [17] Common Query Konstrukte und Entsprechungen in XSLT/XQuery 32
Abb. [18] XSLT-Stylesheet zu Strukturanpassung 34
Abb. [19] Parametrisiertes XSLT-Stylesheet zur Query-Anwendung 37
Abb. [20] Klassenarchitektur der Implementation 39
Abb. [21] Auszug CommonQuery.java 42
Abb. [22] Auszug Boolean.java 43

4 Fehlermeldungen / Fehlercodes

Die XML-Wrapper Applikation bietet zwei Mechanismen zur Behandlung von Fehlern und Ausnahmen.
Während das Programm arbeitet, werden Statusmeldungen (z.B. die Eingabeparameter) und Fehlermeldungen in der Log-Datei „xmlwrapper.log“ mitprotokolliert. Diese Datei wird, falls sie noch nicht existiert, im Verzeichnis erzeugt, in dem sich auch das Programm befindet. Anderfalls werden die Informationen an eine bereits vorhandene Logdatei angehängt. Zur besseren Identifikation bei späteren Kontrollen wird jeder Eintrag mit einem Timestamp versehen.

Zusätzlich sollte auch der Information Broker über Erfolg oder Misserfolg des Wrappers informiert werden können. Aus diesem Grund werden bei einem erfolgreichen Abschluss oder einem Abbruch Systemmeldungen in Form von Zahlencodes erzeugt, die er geeignet behandeln kann.

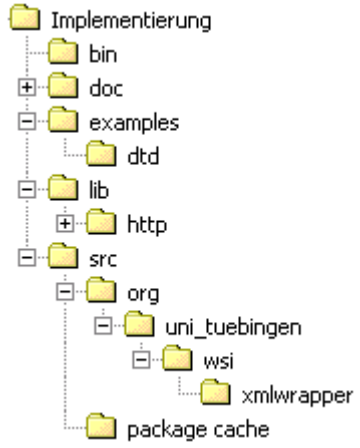
In der folgenden Übersichtstabelle sind die möglichen Fehlercodes, ihre Bedeutung und die Entsprechungen innerhalb der Logdatei aufgelistet.

Systemcode	Erzeugungsgrund	Logdatei-Text
0	Die Arbeit des Wrappers wurde erfolgreich abgeschlossen.	*„XMLWrapper exits successfully.“*
-1	Es wurde die falsche Anzahl von Kommandozeilenparametern übergeben.	*„Error occurred due to invalid number of parameters*
-10	Dateidownload wurde von FTP-Server abgelehnt (z.B. falsches Passwort, ungültiges Verzeichnis)	*„ FTP-Server * refused download connection“*
-11	Download von FTP-Server wurde wegen falscher Authentifikation abgelehnt.	*„unable to login to * with given authentication"*
-12	In URL verwendetes Protokoll wird nicht unterstützt.	*„ * is an unsupported protocol"*
-13	Dateiupload wurde von FTP-Server abgelehnt (z.B. falsches Passwort, ungültiges Verzeichnis)	*"FTP server * refused upload connection"*
-14	Upload von FTP-Server wurde wegen falscher Authentifikation abgelehnt.	*„unable to login to * with given authentication"*
-2	Die in einem Parameter angegebene URL ist ungültig oder konnte nicht erreicht werden.	*„Error occurred while trying to connect to parameter URL [URL].“*
-3	Bei der Abarbeitung der Konfigurationsdatei ist ein Fehler einer Fremdkomponente aufgetreten (wird weiter spezifiziert).	*„Error occurred while accesing SourceDescriptor [reason]*
-30	Bei der Abarbeitung der Konfigurationsdatei ist ein Syntaxfehler aufgetreten.	*„root element of source descriptor is not <XMLSourceDescriptor>"*
-31	s.o.	*"source descriptor needs <source_url> element"*
-32	s.o.	*"source descriptor needs <infounit> element"*

Systemcode	Erzeugungsgrund	Logdatei-Text
-33	s.o.	*"source descriptor needs non-empty <infounit> element"*
-34	s.o.	*"source descriptor needs <resultdtd-url> element"*
-35	s.o	*"source descriptor needs non-empty <resultdtd-url> element"*
-36	s.o.	*"source descriptor needs <properties> element"*
-37	s.o.	*"<property> is missing * attribute"*
-38	s.o	*"<property>'s outer attribute must be 'true'/'false'"*
-4	Bei der Abarbeitung der Common Query ist ein Fehler in einer Fremdkomponente aufgetreten (wird weiter spezifiziert).	*"Error occurred while accessing CommonQuery [reason]“*
-40	Bei der Abarbeitung der Common Query ist ein Syntaxfehler aufgetreten.	*"root element of query is not <query>"*
-41	s.o	*<query> holds more than one term or boolean child element"*
-42	s.o.	*"* is an invalid child of <query>"*
-43	s.o	*"term elements must be empty"*
-44	s.o.	*"term is missing required * attribute"*
-45	s.o.	*" * can not be value of attribute *"*
-46	s.o	*„field * in common query has no counterpart in source"*
-47	s.o.	*„relation * only allowed with truncation='none'"*
-48	s.o.	*"<boolean> must have 2 children"*
-49	s.o.	*" * is an invalid child of <boolean>"*
-50	s.o	*"<boolean> is missing an 'op' attribute"*
-51	s.o.	*" * can not be value of attribute op in <boolean>"*
-6	Beim Mapping der Common Query nach XSLT ist ein Fehler in einer Fremdkomponente aufgetreten (wird weiter spezifiziert).	*"Error occurred while mapping CommonQuery to XSLT [reason]“*
-7	Beim Schreiben des Ergebnisdokuments ist ein Fehler aufgetreten (wird weiter spezifiziert).	*"Error occurred while writing output to [URL] [reason]“*

5 Dateiverzeichnis

Diese Dateien befinden sich auf der beiliegendeneine CDROM bei. Diese enthält folgende Dateien:



: enthält Ausarbeitung als PDF und Bildmaterial
: enthält XML-Wrapper-Programm als JAR-Archive und Batchdatei
: enthält Javadoc-Dokumentation des Paketes
: enthält die verwendeten Datenquellen / Konfigurationsdateien
: enthält DTDs des XMLWrappers und der Beispieldatenquellen
: enthält alle benötigten Fremd-APIs als JAR-Archiv oder als Klassen
: enthält Package-Struktur des XML-Wrapper mit dem Sourcecode und übersetzten Klassendateien
: enthält zur für das JAR-Archives benötigte Dependency-Files

[Fragen und Anmerkungen zur Ausarbeitung an robert.soesemann@student.uni-tuebingen.de]